Manuscript Formatting Guidelines

- Font Size and Style. An Arial typescript must be used throughout the manuscript. The alignment is justified and one-point-five-spaced (use the Document Styles). **Single indention** for every paragraph.

- Paper Size: A4

- Tables and Figures. The tables and figures should fit within the margins if they are part of the main text.

- Margins and Page Numbers Placement. Margins and page number placement should be as follows:
  - Left Margin: 1.5 inches
  - Top, Right and Bottom margins: 1 inch

- Pagination: All pages of the manuscript should be numbered, including the references pages and the appendices. Page numbering with Arabic numbers (1, 2, 3, etc.) should begin with the first page and continue through the end of the paper.

- Footer: The title must appear at the end part of the paper. Font size (minimum 11pt) can vary depending on the length of the title.

- Reference Citation and Style. An IEEE citation style and reference style format must be used in the manuscript.

- Use the appropriate documents styles in your manuscript template.
  - Chapter Content - Chapter Titles
  - Chapter Content - Body
  - Chapter Content - Section Heading
  - Chapter Content - Subsection Heading 1
  - Chapter Content - Subsection Heading 2
  - Chapter Content - Appendix Heading

- Insert appropriate Table and Figure Labels. **References > Insert Caption > Label Type**

- Ensure that Table Headers is continuous/repeating on a page. **Select/Highlight Table Header > Navigate to Table Layout Tab > Select "Properties" > Select "Row" Tab > Check options "Repeat as header row at the top of each page"**

- Update Table of Contents, List of Tables and List of Figures pages. **Navigate to Pages (Table of Contents/List of Tables/List of Figures) > Right-click > Select "Update Field" > Choose "Update Entire Table"**

- Update all Table and Figure numbering in every chapter, section and sub-section as needed. **Select "Table" or "Figure" Caption Number > Right-click > Select "Update Field"**

Other Reminders

- ✓ All explainer section (RED FONT) and example (BLUE FONT) should be removed.
- ✓ **Remove/Exclude this page once the manuscript is finalized or printed**.
- ✓ Next page is a full cover page (image).

QUBO

A Retrieval-Augmented Generation powered Chatbot for Supporting Student Learning through Lecture Notes

QUBO: A RETRIEVAL-AUGMENTED GENERATION POWERED CHATBOT FOR SUPPORTING STUDENT LEARNING THROUGH LECTURE NOTES

A
Thesis
Presented to the Faculty of Computer Science
College of Computing and Information Technologies
National University - Manila

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Science with
Specialization in Machine Learning

BY

Brix Anthony Manzanero

Christian Bongao

Kris Brian Diaz

Miguel Raphael Layos

Renz Andrei Alis

Marizkays Jamison
Thesis Adviser

October 2025

ABSTRACT

ACKNOWLEDGEMENT

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF EQUATIONS

CHAPTER 1
INTRODUCTION

In today's digital age, the massive amounts of information students encounter often lead to significant difficulties in their learning journey, with many struggling due to various forms of insufficient preparation, such as information overload and lack of effective organization which can hinder their overall academic progress [1],[2].

Though digital materials such as lecture notes are provided to students with detailed information about the topic, when it is complex and difficult to navigate, it may hinder rather than support effective learning [3]. Cognitive Load Theory (CLT) explains that learners have a limited working memory capacity, when educational materials are too lengthy or poorly organized, they create unnecessary cognitive load, hindering students' ability to process and retain information [4],[5].

Because of the modern challenges students faces along with the complexity of understanding academic materials, Research has shown that students started to rely on Artificial Intelligence (AI) powered chatbots to help with learning, rather than understanding the source material provided by the universities [6]. A study by Tyton partners indicates that 2 out 3 students use AI chatbots weekly for their studies, and that those with low confidence in their academic standings are more likely to use AI [7].

AI has seen a massive development With The emergence of large language models (LLMs) Which produces text that resembles human writing [8]. Because of the intelligence of these new technologies, students increasingly rely on them, as they offer real-time support to clarify doubts and enhance their learning experience [9].

However, while LLMs have been beneficial in providing learning support for students, they present problems and limitations. They rely on the data they learn from [10]. If the data is outdated, incomplete, or not aligned with the specific educational goals and values of the university, the information provided by these models may be inaccurate or not suitable for the academic context. These models can also hallucinate wherein they generate responses that seem correct, but when investigated deeply it is wrong [11]. This can lead to confusion or misinformation for students who depend on AI tools for their studies. Which can hinder their learning outcomes and affect their knowledge of the profession they are studying.

It is crucial to mitigate these issues to help it align to the educational goals of universities. especially as students increasingly rely on these tools for their studies. By minimizing these issues, AI can provide a safer and more reliable learning experience, supporting students in acquiring accurate knowledge and developing their professional skills [12].

One of the techniques that has the potential to mitigate the issues of hallucinations and lack of contextuality of large language models is Retrieval-Augmented Generation (RAG). This architecture enhances the LLMs capabilities by first using Information Retrieval (IR) to retrieve information coming from trusted sources. By grounding the LLM responses in these sources, it therefore aligns it with the authoritative content, minimizing the chances for hallucination and helping the model produce relevant and reliable responses [13].

RAG systems show promising enhancements in educational contexts, it's essential for students to access precise, contextually appropriate information that aids their learning and aligns with their universities education goals and values [14]. But despite the abundance of studies aimed at improving LLMs via RAG, investigation into its real-world use in education, especially in assisting students to comprehend and learn from their university lecture notes, is still lacking.

To address this gap, we introduce Qubo, a retrieval-augmented question chatbot designed to support student learning through accurate and contextually grounded responses. The system relies on RAG to integrate a knowledge base wherein it seeks relevant information that helps it retain factual consistency to ensure students still retain knowledge from the sources. This study covers the design and development of Qubo. We aim to investigate its effectiveness in providing accuracy and reliability in providing learning support and assess whether this AI-driven system can be a reliable alternative form of learning method for students.

1.1 BACKGROUND OF THE STUDY

Before the pandemic the researchers were already studying how burnout affected university students in low and middle-income nations. Out of 55 total studies, 53 were conducted prior to COVID-19, and they revealed that around 12.2% of students were experiencing complete burnout. Many of these students experienced emotional fatigue, negativity, and an absence of accomplishment, showing that academic stress alone was sufficient to make an important impact [15].

The COVID-19 pandemic highlighted the critical importance of effective educational support, as many students struggled with time management, mental health, and maintaining social connections. Research shows that during periods of remote learning, students' motivation, academic performance, and preparedness for the workforce declined significantly [16]. While some learners improved their digital skills, developed independence, and gained a greater appreciation for teachers, these benefits did not fully offset the difficulties faced by disadvantaged students and the graduating class of 2023, who were among the most severely impacted. In-person learning opportunities were found to alleviate some of these challenges, but variables such as self-confidence, flexibility, and mental health remained central to academic success [17]. Moreover, although lecture notes are typically provided to assist students in understanding course content, they are often unstructured and overly complex, which increases students' cognitive load and hinders effective learning. As a result, many students seek alternative forms of academic support, such as digital tools and interactive platforms, to supplement traditional materials and improve comprehension [18].

Artificial intelligence (AI) is transforming modern education, with AI-driven algorithms becoming essential components of learning management and training systems. Findings indicate that AI could significantly increase students' productivity and make learning more accessible [19]. However, researchers have raised concerns about reduced human interaction, threats to data privacy, algorithmic bias, and unequal access to technology. It is important to have an effective approach that integrates technological advances with ethical standards, educator training, and transparency for maximizing the advantages of AI while minimizing the potential risks [18].

Because of the academic pressures and cognitive burdens inherent in modern education, students have embraced the use of generative artificial intelligence with unprecedented speed and scale. The emergence of powerful, publicly accessible Large Language Models (LLMs) like ChatGPT has created a paradigm shift in how students approach their academic work [20],[21]. As of 2025, an overwhelming majority of students in higher education, ranging from 86% to 92%, report using AI tools for their studies. This trend has shown explosive growth, with AI usage among university students surging from 66% in 2024 to 92% just one year later [22]. ChatGPT stands as the undisputed leader in this space, with 66% of students globally using it for coursework, making it the most common tool by a significant margin. The

technology's rapid ascent is further evidenced by its record-breaking user acquisition, reaching 100 million monthly active users just two months after its launch [23].

However, this rapid adoption of LLM-powered chatbots is a double-edged sword because, with its benefits, come new challenges. These powerful technologies can produce errors and misinformation. AI models are trained to generate plausible text, not to verify facts, and the data they draw from may be outdated, contain errors, or perpetuate misinformation. One manifestation of these errors is called hallucination, where the model generates responses that appear plausible but are inaccurate or false [11], [24]. This concern is especially critical in education, where knowledge must be precise. This issue of hallucination in LLMs can be crucial in some scenarios and tasks because it can cause harm. However, it is widespread across different models, with causes that are multifaceted. Current evaluation benchmarks often fail to capture hallucination effectively, and mitigation strategies are promising but still immature [13].

A second, equally significant limitation is the static nature of an LLM's knowledge base. Since LLMs rely on the data they were trained on, they cannot capture data sourced from private universities. A student in a political science course cannot ask about the results of a recent election [18]. A medical student cannot get information on the latest clinical trial results. Even within a single semester, the LLM would be ignorant of course-specific announcements, updated reading lists, or clarifications provided by the instructor in a recent lecture. This lack of context might misalign students with the learning outcomes being instilled by the universities.

In response to the inherent limitations of Large Language Models, Retrieval-Augmented Generation (RAG) has emerged [14]. It is a redesign of how LLMs collect information. It works by grounding responses based on reliable sources by connecting the model to dependable, current information sources instead of relying only on what the model learned during training, using information retrieval with a large language model. With this approach, instead of abandoning the vast amount of information that academic materials like lecture notes contain, they can be compiled in a storage system, then retrieved as important documents based on what is needed at the time. The documents can then be used by the Large Language Model to answer the question.

According to initial studies, RAG-powered chatbots not only generate more accurate responses but also perform well in learning environments by providing reliable, course-specific assistance [13], [19]. According to researchers, this method

helps address several issues with traditional LLMs, such as lack of transparency, outdated knowledge, and hallucinations [20]. But even with research aimed at improving RAG, practical applications for helping students learn are still lacking.

Considering these challenges, this study aims to develop a chatbot that focuses on helping students in learning from lecture notes. Qubo is a chatbot designed to support student learning by leveraging lecture notes through Retrieval-Augmented Generation (RAG). It aims to equip students with additional tools for self-learning. These types of tools can maintain student interest and encourage self-learning. Qubo, addresses the growing need for effective digital support systems. This is particularly significant in the Philippines, where access to high-quality educational resources remains limited despite the strong cultural value placed on education [25]. AI-powered platforms such as Qubo demonstrate how smart integrations of advanced technologies can reduce students' workload, simplify the learning process, and promote more equitable access to knowledge.

1.2 STATEMENT OF THE PROBLEM

The learning challenges faced by students in the digital age, such as information overload and lack of effective organization can hinder their overall academic progress. The lengthy and complex educational material such as lectures notes provided by universities can hinder learning than be an effective tool because of the increased cognitive students faced when trying to understand these resources. Because of these issues. There is an increase in students' reliance on AI chatbots to help them with learning.

But these AI chatbots powered by large language models can have outdated information or not aligned with the universities learning goals and values, furthermore they generate hallucination response wherein it may seem correct but is false when investigated further, making them unreliable and unstable for use in educational contexts, where factual, and accurate information is crucial for learning. The students relying on these tools for academic support could suffer from a distorted understanding of the subject matter, leading to potential knowledge gaps, incorrect conclusions, or even poorer academic performance. That said, these problems highlight the need for reliable and contextually grounded learning assistive tools that can help students effectively navigate their course materials and improve their learning outcomes.

1.3 OBJECTIVES OF THE STUDY

The general objective of this study is to design and develop Qubo, a Retrieval-Augmented generation powered chatbot that supports student learning from their lecture notes by providing accurate, contextually grounded, and reliable learning support.

The specific objectives are as follows:

1. To develop a retrieval-augmented generation pipeline capable of processing and understanding lecture notes as the primary knowledge source.

2. To develop effective retrieval methods that improve accuracy, contextual comprehension, and adaptability to diverse lecture note formats.

3. To develop strategies for improving a Large Language Model response generation, focusing on alignment with retrieved content.

4. To evaluate Qubo's reliability, accuracy, and usability through comprehensive system testing and student assessments, to determine the platform's potential as a scalable and effective supplemental learning tool.

1.4 SIGNIFICANCE OF THE STUDY

The results from the development and evaluation of this study will benefit the following:

For Students, Qubo provides an efficient and accessible method for students to obtain timely learning support. The system addresses challenges associated with lengthy and complex lecture notes by delivering precise, course-specific answers instantly. This approach facilitates comprehension of difficult concepts without requiring students to review extensive materials. By reducing cognitive load, Qubo mitigates frustration and disengagement. Additionally, Qubo functions as a continuously available tutor, offering support beyond faculty office hours and without the financial or scheduling constraints of traditional tutoring. Students can learn at their own pace and assume greater responsibility for their academic progress.

For Educators and Institutions, the adoption of Qubo provides several strong practical and intellectual benefits to faculty members and to the university. Individually, for teaching staff and assistants, Qubo significantly reduces time spent dealing with commonly queried, repetitive questions about courses. Optimizing tasks in this way allows teachers to direct their energies to higher order activities, such as creating high-

standard teaching materials, creating lively classroom discourse, and providing thorough, individual feedback that only a human can give. Secondly, for institutions, this work provides a proven model of how to incorporate trustworthy AI tools into their online learning environments. Adoption of Qubo demonstrates a commitment to providing good-quality resources that are trustworthy and accurate, enhancing student satisfaction and giving the university's e-learning facilities a confidence boost.

For Future Researchers, this work contributes to further development of AI in teaching environments through a practical application of the RAG technique to enhance the reliability of AI teaching tools. This work provides an all-inclusive plan of action for developing credible AI tutors that go beyond abstract intellectual discussions. Researchers aiming to enhance the accuracy of school technology can use methodologies provided to link AI systems to specific knowledge bases, such as lecture notes. By demonstrating how Qubo addresses the problem of disinformation produced by AI systems, this work provides a foundation upon which to build other credible AI tools in a wide range of subjects and teaching environments.

1.5 SCOPE AND DELIMITATION

1.5.1 Scope of the Study

The purpose of this study is to create a RAG chatbot that will help students in asking enquiries using academic lecture notes. The system's primary function is to allow students to upload digital lecture notes which are eventually processed into a knowledge base for retrieval and response generation. The chatbot uses a retrieval pipeline and an instruction tuned Large Language Model to generate responses that are grounded in the uploaded documents. It can handle file types such as PDF, DOCX, and plain text documents with varying structures such as bullet points, paragraphs, or tables. The system is also capable of extracting text from the images within PDF and DOCX files. The system also supports different forms of documents from simple personal notes to academic lectures.

This study covers the design, development, and testing of chatbot as a prototype learning tool. It highlights basics such as document intake, Information retrieval, and natural language answer development, making it helpful in both the classroom and self-study settings. The system's target audience are students who regularly use lecture notes as their primary learning resource.

1.5.2 Delimitations of the Study

This study was conducted within the constraints of available resources, including time, budget, and computational capacity. As a result, the implementation focuses on a feasible and manageable approach suitable for a prototype or proof-of-concept. While this allows for the demonstration of the core concepts, it may limit scalability, performance, or the inclusion of more advanced features. These aspects are acknowledged as potential directions for future work.

The platform has limitations in processing non-textual elements, Although the platform can handle text extraction from documents images, it does not have the ability to get the context from a non-textual image as it cannot support image recognition. Additionally, it struggles with complex documents like research papers, which contain specialized terminology and intricate structures. These limitations are due to the constraints of the current prototype, which is designed to focus on text-based content. Handling more advanced documents would require further development in natural language processing and additional computational resources, which are beyond the scope of this study.

CHAPTER 2
REVIEW OF RELATED LITERATURE AND STUDIES

2.1 RELATED LITERATURES

2.1.1 Challenges in Learning

Students face multiple challenges in learning such as lack in motivation and engagement especially in a digital learning setup since these setups expose students to distractions such as social media which can shift focus in studying. When unmotivated, students engage superficially with the learning materials by memorizing instead of having a deep cognitive strategy, which can have effect on retention and limitations to apply the knowledge in more complex contexts [26], [27].

The vast workload of student's experience can add to a heavy burden that can be stressful and challenging to learn effectively in each subject [26]. Although Academic resources such as lecture notes can aid in learning the subject. Poorly structured lecture notes or fragmented instructional materials can overwhelm students. Research of Cognitive Load on Learning Memory in Accounting Students in the Philippines emphasizes that both intrinsic and extraneous cognitive load must be reduced to improve learning efficiency. Poorly structured lecture notes or fragmented instructional materials can overwhelm students' working memory, making it harder to process and retain new knowledge. In contrast, well-designed learning materials that minimize extraneous cognitive demands can enhance comprehension and support long-term memory retention [5] [26].

2.1.2 The Role of AI in Student Learning

AI is reshaping education by transforming how students learn and has the potential to address some of the biggest challenges in education today [29]. Increase AI-driven tools are being adopted by institutions in higher education, to keep pace with innovation and prepare students for a rapidly changing world [30]. At the core of this transformation are AI subfields such as machine learning (ML) and natural language processing (NLP)[8].

With the rise of LLMs, AI is being used by students to help with understanding lessons within their schools. This individualized approach helps close learning gaps, enhances engagement, and improves academic outcomes. Not only that teachers benefit from AI by automating repetitive tasks such as grading and attendance,

allowing them to focus on higher-value instructional activities while ensuring students still receive consistent and actionable feedback [31]. AI facilitates data-driven insights into student performance, enabling educators to monitor progress more effectively and adjust teaching strategies in real time. This is particularly valuable in contexts with large or diverse classrooms, where individual monitoring is difficult [32].

In the Philippine educational context, AI integration presents both opportunities and challenges. Local researchers highlight the potential of AI to enhance personalized learning, support teachers in designing effective instruction, and provide scalable educational solutions [33]. However, issues such as privacy, data security, and the risk of widening the digital divide must be addressed to ensure equitable access and ethical implementation, that is why it had led to new ethical policies [34].

2.1.3 Chatbots Powered by Large Language Models (LLMs) in Student Learning

Chatbots have been a vital tool for student learning because of their capability of on demand support. These systems improve student learning by answering questions immediately with detailed explanations and can adapt to different learning styles. This is why many students find these systems to be helpful because of their ability to clarify misunderstandings or add clarify misconceptions when there is no support from instructors [20] [35].

Chatbots have existed for a long time but are limited in capabilities, it is only in 2020 wherein it had seen massive improvements due to emerging breakthroughs in Deep Learning Architectures, specifically through the development of LLMs [36]. One of the key strengths of LLM-powered chatbots is their ability to match responses to the learner's level of knowledge, ensuring that explanations are appropriately pitched and relevant. This helps in classrooms with a range of competencies, so that weaker students are supported without slowing down stronger ones. Moreover, chatbots can handle repetitive questions or frequently covered topics, freeing up instructors to focus on more complex instructional tasks [8].

However, chatbots powered by large language models also face hallucinations issues wherein they generate responses that sound plausible but are factually incorrect or misleading. Another challenge is that while chatbots excel at delivering information and clarifications, they may be less effective in facilitating higher-order skills such as critical thinking, deep conceptual reasoning, and dialogic discussion.

Thus, while LLM-based chatbots can be powerful tools, they are better used in conjunction with traditional instruction to ensure reliability and depth of learning. [37].

It is necessary to determine the appropriate conditions and methods for the use of chatbots to eliminate the disadvantages that will help AI powered chatbots to transform education as chatbots have the advantages of increasing student performance and motivation, decreasing instructor workload and improving educational resources [12], [37].

2.1.4 Information Retrieval

Information Retrieval (IR) is a field of study that deals with the organization, storage, and retrieval of information from a large collection of documents. This discipline is used in creating systems like library catalogs and web search engines. Traditional Information retrieval systems work by storing many documents or data sources, like web pages, books, research papers, or any text-based information, which are stored efficiently [38]. And when a user asks a question, it searches for it using matching functions to search in the collection of documents to find information that can answer the question, which is usually ranked to provide the user with the most relevant information stored in the system [39].

Traditional information retrieval methods before uses Term Frequency- Inverse Document Frequency(TF-IDF) and Best Matching 25(BM25) which relies on statical metrics to determine relevant of documents to a query, For instance, TF-IDF analyzes the frequency of terms in documents and queries, while BM25 improves this approach by considering term saturation which is why both these methods have been widely adopted due to their simplicity, efficiency, and proven robustness in retrieving relevant information [38] [39].

Because of the increasing information in the modern age, current information retrieval systems are using advanced techniques like natural language processing, machine learning, and algorithms to rank search results and present the most useful information to user [40]. Such as generating embeddings using Sentence-BERT models that turn them into vector representations that capture semantic similarity between documents and queries. This enables semantic search, allowing the retrieval of documents even when exact words do not match, as long as their meanings are contextually related. Similarity can then be measured using methods such as dot product search or cosine similarity [41][42].

Because of the increasing reliability of modern information retrieval, researchers have explored how Information Retrieval can be utilized to support LLMs, to capture relevant information to help improve its outputs accuracy and contextual comprehension to mitigate the problem of hallucinations, which is crucial in the education sector, where accuracy and trustworthiness of information are vital [40], [42], [43].

2.1.5 Retrieval-Augmented Generation

RAG has emerged as a solution to mitigate some of the challenges faced by LLMs, particularly hallucination and limited knowledge generalization. It is a hybrid approach that enhances the LLMs capabilities by enabling it to retrieve information from external sources with its text generation [7], [44], [45]. RAG is an architecture designed to enhance LLMs by first searching for useful information before responding, rather than answering solely based on the LLM's learned knowledge from training. LLMs are trained in vast volumes of data and use billions of parameters to generate original output for tasks like answering questions, translating languages, and completing sentences. RAG can extend the already powerful capabilities of LLMs by grounding its responses in specific domains or an organization's internal knowledge base without retraining the model [46]. RAG models can ground outputs in reliable documents, ensuring responses are both relevant and verifiable, which could be beneficial for dealing with educational materials.

Figure 1 illustrates the basic process of how LLMs work. When a user asks a question, the LLM (e.g., GPT, LLaMA) takes the question as input and processes it through multiple attention layers. These layers help the model understand how words relate to each other and predict what words should come next based on patterns learned from its training data. The input is refined across several layers until it reaches the final output layer, where the model generates its response to the user. With this the generation of answers relies solely on the data learned by the model [47].

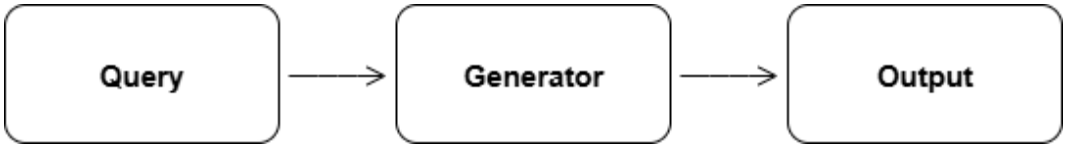

Figure 1. Basic LLM Process

Figure 2 illustrates how RAG enhances the functionality of LLMs, following the framework proposed by researchers [7], [44]. The approach can be broken down into three main stages:

1. Indexing: Documents are preprocessed, cleaned, and divided into chunks, often with semantic meaning preserved. Each chunk is vectorized using embeddings such as BERT or Sentence-BERT and stored in a vector database along with optional metadata.

2. Retrieval: When a query is received, it is vectorized and compared to document vectors using similarity metrics like cosine similarity. The most relevant documents are retrieved to provide context for response generation.

3. Generation: The LLM uses the retrieved documents in conjunction with its pre-trained knowledge to generate a response. This ensures output is both fluent and coherent while grounded in specific, relevant information. The retrieved context improves accuracy and relevance, while the LLM's language capabilities maintain natural tone and style.

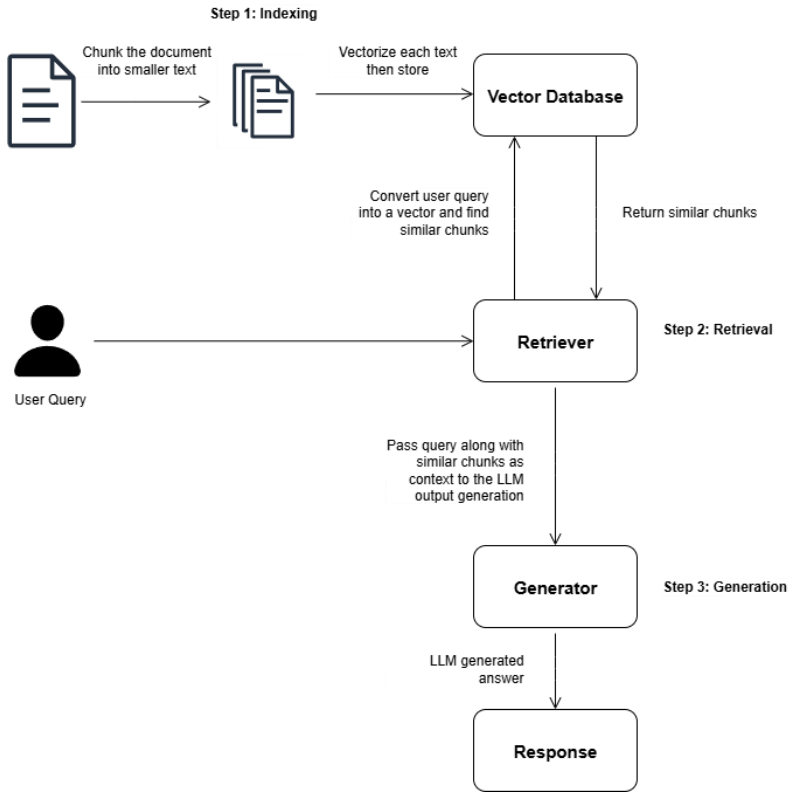

Figure 2. Basic Retrieval-Augmented Generation Process

2.1.6 Advances in RAG

Recent developments in Retrieval-Augmented Generation (RAG) focus on making Large Language Models (LLMs) more contextual, reliable, and academically useful. Traditional LLMs often suffer from issues like hallucinations, outdated information, and lack of source referencing. RAG addresses these problems by integrating retrieval systems that ground responses in authentic materials such as lecture notes and study guides [48], [49].

Advances in RAG research have improved both retrieval and generation. Retrieval has evolved from simple keyword matching (sparse retrieval) to dense and multi-stage retrieval, which better understands meaning and reformulates queries to find more relevant information—especially useful for students asking complex questions.

Modern fusion strategies now use context filtering and adaptive techniques to highlight the most relevant content, producing clearer and more focused responses. Additionally, new RAG systems incorporate reasoning capabilities, enabling them to combine insights from multiple documents and link related academic concepts across topics [50],[51].

Recent studies also emphasize retrieval diversity including some lower-ranked or noisy documents to increase robustness and stimulate curiosity and deeper learning. Moreover, RAG advances now target domain adaptation and privacy-preserving retrieval, ensuring that educational chatbots stay within instructor-provided materials and protect sensitive data [52],[53],[54].

In summary, the evolution of RAG has made it an ideal foundation for educational chatbots. By enhancing retrieval accuracy, contextual reasoning, and data security, RAG-based systems can provide grounded, coherent, and course-specific assistance that promotes deeper understanding and independent learning.

2.1.7 React and Fast API as an app development stack

React is a declarative, component-based library that allows developers to build reusable UI components. Its modular architecture enables the creation of highly interactive and responsive user interfaces that update dynamically as data changes, while preserving a clear separation of concerns. These characteristics make React

particularly well-suited for applications where engaging user experience is a priority [55].

FastAPI is a high-performance Python web framework based on Starlette and Pydantic. It maximizes speed and developer productivity by offering automatic data validation, type annotations, and interactive API documentation via OpenAPI and Swagger. The asynchronous architecture supports efficient handling of concurrent requests, which is advantageous for real-time applications and microservices. FastAPI also integrates with the Python machine learning ecosystem, enabling the incorporation of advanced data processing and AI functionalities into applications [56].

The combination of React and FastAPI forms a robust technology stack for contemporary application development. React manages frontend presentation and user interaction, while FastAPI handles backend logic, application programming interfaces (APIs), and data processing. This clear separation of concerns enhances scalability, maintainability, and development speed. The stack is particularly effective for constructing data-driven solutions, including AI-powered chatbot applications.

2.2 RELATED STUDIES

2.2.1 Hallucination in Large Language Models

Recenty LLMs have exponentially improved in fluency due to the development of new solutions such as transformer-based architecture. But attention to their limitations and potential risks has also increased [57]. LLM models are particularly notorious in the generation of outputs that are coherent and plausible but factually incorrect or misleading. This phenomenon, known as "hallucination," has been a growing focus of research [58]. These inaccuracies pose significant issues in educational settings, as students might inadvertently accept misleading information, resulting in incorrect conceptual understanding and lasting misconceptions. This is particularly relevant for students with limited subject matter expertise. They are especially vulnerable to overreliance on AI-generated responses. Without the skills to independently verify information, they may accept erroneous outputs at face value, compounding the problem of misinformation in learning environments [59].

The issues with LLMs like ChatGPT often produce responses interspersed with inaccuracies or fabrications [60]. A study proved this issue by utilizing 50 abstracts sourced from five scientific journals and assigning ChatGPT to create abstracts based on their titles. Subsequently, both sets of abstracts underwent scrutiny by AI plagiarism

detection software and impartial human assessors. The results revealed that out of the abstracts generated by ChatGPT, 68% were correctly identified as such (true positives), while 14% of authentic abstracts were mistakenly categorized as chatbot-generated (false positives). These findings underscore the model's limitations in accurately generating information, which can hinder its ability to provide reliable sources for users [61]. Another study, wherein the authors evaluated the mechanistic reasoning of three dialogue systems (ChatGPT-3.5, ChatGPT-4, and Bard). The findings revealed that chatbot responses either underperform or reason on par with students and occasionally provide inaccurate answers to content questions. The results suggest that dialogue systems exhibit limited mechanistic reasoning capabilities compared to students. This study suggests that students, when equipped with knowledge, can question the reasoning of LLM models, as these models sometimes provide inaccurate answers [62].

Hallucinations occur because of how the models are trained. Language models are first learned through pretraining, a process of predicting the next word in huge amounts of text. Unlike traditional machine learning problems, there are no "true/false" labels attached to each statement. The model sees only positive examples of fluent language and must approximate the overall distribution. This may generate plausible but incorrect information that fits learned patterns. Second, Models are trained on large static data, which may contain inaccurate data or be outdated in time. Third, there is an unequal distribution of what can be used to train the model. Not all knowledge fields can be equally represented, such as topics that are complex, such as flight dynamics or fluid mechanics, where domain-specific data may be scarce or poorly represented in public datasets. And lastly, there is an upper bound knowledge capacity of models. Models use computational power to accommodate resources for more learning. However, there is no model yet powerful enough to fully encode all human knowledge, especially for rare or domain-specific facts. All these can be attributed to why hallucinations persist, and future research may target these factors to minimize hallucinations from LLMs [11], [63].

2.2.2 RAG in Mitigation of Hallucinations

Researchers examined how retrieval augmentation affects hallucination during structured output tasks. In a comparison between a baseline standalone LLM and a RAG-driven solution, they established that the model with RAG produced a significantly lower number of hallucinated items (like invalid properties or steps) during workflow generation. Their findings also showed that not only was hallucination

diminished using RAG but also that generalization to out-of-domain inputs was enhanced with a demonstration of its strength over a standalone LLM running without retrieval aiding [44].

The RAG framework for knowledge-driven dialogue evaluated its ability to reduce unsupported responses in conversational tasks. By rooting the generation process in relevant passages procured from large text corpora, they showed that models of RAG produced more factual, grounded, and contextually specific dialogue compared with independent LLMs. Their findings show the effectiveness of retrieval methods in reducing hallucinations, in situations where factual correctness takes center stage [64].

The effectiveness of RAG systems for hallucination suppression often depends on retrieval quality. Thus, a hybrid system that blends sparse retrieval strategies with dense retrieval approaches was found in Hybrid Retrieval for Hallucination Mitigation in Large Language Models to provide more uniform grounding with lower hallucination rates as well as better performance on HaluBench dataset. It supports prior evidence that factuality directly depends on retrieval quality. Furthermore, more advanced methods can further improve the performance of RAG. DePaC that combats hallucinations in RAG when employing parallel or unordered context requirements. In merging negative training with context signal calibration upon aggregation, they mitigated fact fabrication as well as omission error types and showed improvements on several NLP tasks [65].

Collectively, these results underscore the potential of RAG to alleviate the occurrence of hallucinations in the output generated by LLM across a variety of tasks. Nevertheless, the researchers also emphasize several constraints, including reliance on the quality of retrieval, difficulties in reconciling conflicting sources, and the possibility of residual hallucinations arising when pertinent evidence is absent or misinterpreted.

2.2.3 Challenges and Limitations of RAG

Several studies have examined not only the potential of RAG to improve factual grounding in LLMs but also the challenges that limit its effectiveness. The causes of hallucinations in RAG systems shown in Figure 3 are categorized into two key sources of error: retrieval failures and generation deficiencies. Retrieval failures occur when the system is unable to access or select relevant documents due to weak queries,

flawed retrieval strategies, or ineffective retrievers. To address these issues, scholars have suggested methods such as query refinement, iterative retrieval, and adaptive retrieval. Meanwhile, generation deficiencies arise when retrieved content is misused, leading to irrelevant or incomplete responses. Issues such as context noise, the “middle curse” (in which information in the middle of a passage is overlooked), and alignment errors have been reported. Proposed solutions include context compression, document re-ranking, and faithfulness techniques that aim to ensure output remains consistent with retrieved evidence [66].

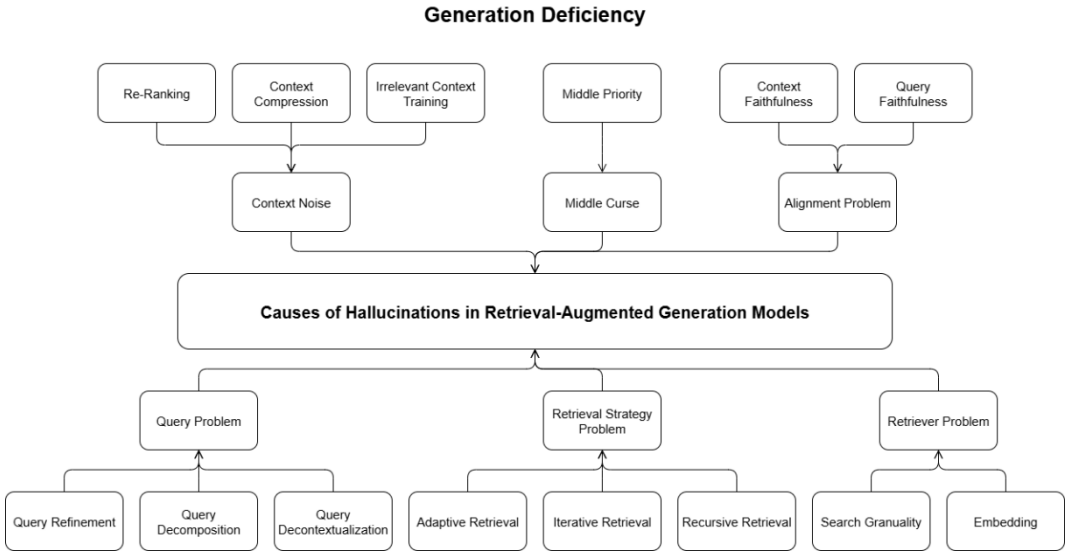

Figure 3. Causes of Hallucinations in RAG Models

Other researchers have also focused on weaknesses in the retrieval mechanism. While retrieval grounds responses in external knowledge, it can perform poorly when queries are ambiguous or domain specific. Dense retrieval approaches, which depend heavily on vector representations, have sometimes been shown to yield irrelevant results [7],[67]. To improve retrieval accuracy, techniques such as query expansion and contextual disambiguation can be explored.

Limitations are also evident in the generation stage. Although RAG is designed to integrate retrieved information with generative reasoning, models often fail to incorporate external knowledge effectively [62], leading to hallucinations or shallow responses. This concern can be addressed by enhanced attention mechanisms and hierarchical fusion techniques to strengthen alignment between retrieved documents and generated outputs [65].

The quality of data retrieved has likewise been highlighted as a crucial factor. Noisy, inaccurate, or incomplete retrieval corpora introduce errors that can mislead generations. Related to this is the challenge of token limitations, since LLMs can only process a fixed number of tokens at once. Studies suggested salience detection and diversity-aware ranking to prioritize the most relevant content and reduce the risks of incomplete or misleading answers [51].

Finally, several works point to computational efficiency as a limiting factor. Compared to stand-alone LLMs, RAG involves multiple stages retrieval, ranking, and generation which can increase latency. This poses challenges for real-time applications. pre-computation, and parallel processing are possible strategies to improve system responsiveness without sacrificing accuracy [65].

Collectively, these studies show that while RAG offers substantial benefits in mitigating hallucinations, its effectiveness depends on resolving persistent challenges such as retrieval accuracy, generation alignment, data quality, token constraints, and computational overhead. Addressing these limitations remains a key focus in the evolving research on retrieval-augmented systems.

2.2.4 Rag applications in Student Learning

Recent studies demonstrate that RAG systems consistently deliver more accurate and contextually specific answers across a variety of tasks. By pulling information from external knowledge bases at inference time, RAG helps LLMs stay current without the need for expensive retraining [51], [62]. As RAG is continuing to advance, notable studies have explored its implementation on helping students learn

For example, a RAG-based academic chatbot designed to enhance information services in higher education. Unlike standard chatbots and standalone LLMs that are prone to hallucinations, their system relied on official knowledge bases to ensure reliability. In testing 100 questions, the chatbot achieved high factual accuracy with a faithfulness score of 0.91, though it performed less strongly on reasoning tasks. Notably, it successfully rejected irrelevant questions 81.25% of the time demonstrating strong safeguards against misuse while highlighting the ongoing challenge of improving reasoning abilities [67].

Building on this direction, BiWi AI Tutor, a RAG-based mentoring system that integrates course materials with LLM capabilities. This tutor achieved up to 87% accuracy, with evaluations from GPT-4 aligning closely with human expert judgments.

Performance further improved when reranking methods were applied, especially in handling school-related questions. While the system illustrates how RAG can enhance digital learning, the authors also noted that more progress is needed in areas such as personalization, ethics, and mentoring-style interactions [68].

Another notable study is the MARK system (Machine Assistant with Reliable Knowledge), designed as a virtual tutor and support assistant for engineering education. MARK uses a combination of keyword, sparse, and dense retrieval techniques with reranking to ensure answers remain grounded in course content. Tested both as an academic tutor and as a technical support tool, it showed adaptability across use cases and improved with feedback. However, its effectiveness still depends heavily on the quality of its knowledge base, and at present it only supports text. Future development may involve multimodal features and more advanced personalization strategies [69].

Together, these studies highlight the promise of RAG-based systems in higher education: they improve factual reliability, enhance access to institutional knowledge, and provide scalable learning support. At the same time, they underline important limitations, especially in reasoning ability, personalization, and long-term integration into classroom practice.

2.3 SYNTHESIS

AI continues to shape the educational landscape, but concerns privacy, data security, and the risk of widening the digital divide must be addressed to ensure equitable access and ethical implementation [34]. Challenges in learning, such as overwhelming workload along with cognitive load from complex information and overloaded educational materials, often push students to rely on chatbots to help with complex topics and assignments. Modern chatbots, powered by LLMs, have seen significant advancements in recent years. Many students who use this technology report improvements in academic performance. However, LLMs also present limitations such as hallucinations, which is a serious concern in the educational context where knowledge must be precise. Because some students rely too heavily on these systems without evaluating the information provided, they risk experiencing unsatisfactory learning outcomes.

To address this, several researchers have explored hallucination mitigation techniques, one of the most prominent being RAG. Researchers have explained its

ability to reduce hallucinations by integrating a knowledge base before generating answers, ensuring that responses are grounded in external data rather than relying solely on the LLM's training data, which may lack context for specific questions or be outdated.

Implementing RAG, however, involves several workflows, such as indexing documents, retrieving relevant materials, and augmenting the LLM to generate responses. Researchers have presented various RAG implementations, resulting in multiple methods and variations. Because of this diversity, RAG must be carefully evaluated based on the task to ensure that it achieves its intended purpose rather than exacerbating hallucinations. When poorly designed, errors in either the retriever or the generator can still lead to hallucinations. Moreover, RAG introduces additional complexity, as developers must optimize each component of the system before deployment.

This is why RAG's practical application in use as a tool for helping students learn remains largely at the prototype stage, with most implementations still operating on a small scale. Nevertheless, its potential is significant. By ensuring retrieval precision such as aligning a retriever with structured sources like lecture notes, developers can create educational chatbots that minimize hallucinations while enhancing reliability. Over time, such systems can be incrementally improved, balancing accuracy, efficiency, and scalability. However, this requires careful planning, as RAG involves multiple workflows, each with its own parameters and implementation techniques.

This opens an avenue to create a precise RAG chatbot aimed at directly grounding responses to build a reliable, transparent, and pedagogically aligned learning chatbot one that grounds every answer in verifiable course materials, clearly cites its sources, flags uncertainty instead of guessing, and supports students' understanding rather than shortcutting it. By leveraging new techniques in retrieval, optimizing the parameters, and using an instruction tuned model to allow carefully constructed prompts, RAG system can reduce hallucinations ground response into institution-owned knowledge bases, and narrow the learning challenges of students by providing a learning support tool that is delivering accurate help on a scale. With careful retrieval design, proper generation guidance, iterative evaluation and ethical safeguards, a precise RAG chatbot can measurably improve learning outcomes while maintaining the trust and integrity required in education.

CHAPTER 3
RESEARCH DESIGN AND METHODOLOGY

3.1 RESEARCH DESIGN

This study will employ a mixed-methods research design, integrating both quantitative and qualitative approaches. The quantitative component focuses on measuring user interactions and learning outcomes through structured surveys completed by participants. The qualitative component gathers in-depth insights from open-ended survey items and follow-up interviews and a system evaluation by an expert in computer science to provide an informed perspective on the system's overall effectiveness. The mixed-methods design is appropriate because it captures both the human-centered experience of users and the expert-informed assessment of the system, ensuring a comprehensive evaluation of Qubo.

3.2 PARTICIPANTS AND SAMPLING TECHNIQUES

The user evaluation will involve 45 undergraduate students enrolled in various courses from different universities. This diverse selection aligns with Qubo's design goal of serving as a general-purpose learning tool applicable across disciplines.

A convenience sampling technique will be used to recruit students, considering practical constraints such as time and accessibility. Students will participate in scheduled sessions, interact with Qubo, and provide feedback immediately afterward through structured surveys and, for selected students, follow-up interviews.

In addition, the system will be evaluated by one professional expert in computer science or educational technology. The expert will provide a qualitative assessment of Qubo's accuracy, reliability, and overall usability. This evaluation complements the student feedback by offering an informed, expert perspective on the system's performance and suitability for learning.

3.3 DATA GATHERING METHODS

Data for this study will be collected through user evaluation, focusing on participants' interactions and experiences with Qubo.

After completing guided learning tasks, participants will provide feedback through:

- Structured surveys consisting of Likert-scale and open-ended items, measuring usability, learning outcomes, and satisfaction.

- Follow-up interviews with selected students to gather more detailed qualitative insights into their experiences and perceptions of the system.

In addition, a professional expert in computer science or educational technology will evaluate Qubo. The expert will provide a qualitative assessment of the system's accuracy, reliability, and overall usability. This expert evaluation is included to complement student feedback, ensuring that the system is not only usable from a learner's perspective but also meets professional standards for correctness and functionality.

By combining student-centered data and expert evaluation, this approach captures both quantitative measures (from surveys) and qualitative insights (from interviews and professional assessment), providing a comprehensive understanding of Qubo's effectiveness and user experience.

3.4 RESEARCH INSTRUMENTS

3.4.1 Survey Questionnaire

The survey will consist of two parts: (1) ten Likert-scale items measuring usability, reliability, clarity, and educational impact, and (2) four open-ended questions allowing participants to describe strengths, challenges, and suggestions for improvement. The survey is designed to capture both quantitative measures and qualitative insights. The survey content is as follows:

Table 1. Survey Questionnaire

	Statement	1 Strongly disagree	2 Somewhat disagree	3 Neither agree nor disagree	4 Somewhat Agree	5 Strongly agree
1	Qubo was easy to use and navigate.					
2	The responses generated by					

	Statement	1 Strongly disagree	2 Somewhat disagree	3 Neither agree nor disagree	4 Somewhat Agree	5 Strongly agree
	Qubo were clear and understandable.					
3	Qubo provided accurate information that matched the lecture content.					
4	Using Qubo improved my understanding of the subject matter.					
5	Qubo supported my independent study more effectively than traditional notes.					
6	Qubo responded in a timely manner when completing tasks.					
7	The interaction with Qubo kept me engaged in the learning process.					
8	I would recommend Qubo to other students as a learning tool.					
9	Qubo helped me recall and review					

	Statement	1 Strongly disagree	2 Somewhat disagree	3 Neither agree nor disagree	4 Somewhat Agree	5 Strongly agree
	important lecture concepts.					
10	Overall, I am satisfied with Qubo as a supplemental learning tool.					

In addition to the Likert-scale items, participants will also respond to the following open-ended questions:

1. What aspects of Qubo did you find most helpful in supporting your learning?
2. What difficulties did you encounter while using Qubo?
3. How does Qubo compare to your traditional study methods?
4. What improvements would you suggest for Qubo?

3.4.2 Interview Guide Questions

To gain deeper insights, semi-structured interviews will also be conducted with selected participants. The guide's questions will focus on students' personal experiences with Qubo, specifically:

1. Can you describe a specific instance where Qubo helped you understand a topic better?

2. Were there moments when Qubo's responses were confusing or unhelpful? Please explain.

3. How did you feel using Qubo compared to studying on your own or with classmates?

4. In what ways could Qubo be improved to better support your learning needs?

3.4.3 Expert Evaluation Guide

To complement student feedback, a professional expert in computer science or educational technology will evaluate Qubo. The expert will provide a qualitative assessment of the system's Accuracy, Reliability, and Usability.

The expert will be guided by a set of semi-structured evaluation prompts, such as:

1. How accurate and reliable are Qubo's responses compared to standard lecture content?
2. Are there any instances where Qubo's responses may be misleading or incomplete?
3. How user-friendly is Qubo in terms of navigation and interaction?
4. What improvements would you suggest enhancing the system's effectiveness for learners?

This instrument ensures that the system is reviewed from an expert perspective, providing insights into both functional correctness and educational suitability.

3.5 DATA ANALYSIS TECHNIQUES

Data collected from this study will be analyzed using both quantitative and qualitative methods.

3.5.1 Quantitative Analysis

- Student responses to the Likert-scale survey (1–5) will be analyzed using descriptive statistics.

- The mean will be calculated for each item to summarize participants' perceptions, using the formula:

$$\text{Mean} = \frac{\text{Sum of Data Points}}{\text{Number of Data Points}}$$

Equation 1. Mean Formula

3.5.2 Qualitative Analysis

- Responses from open-ended survey questions and student interviews will undergo thematic analysis. The process involves:
    - **Familiarization**: Reading and understanding the responses.
    - **Coding**: Identifying initial codes representing key ideas.
    - **Categorization**: Grouping similar codes into broader categories.
    - **Theme Identification**: Extracting emerging themes such as strengths, challenges, and suggestions for improvement.
- The **professional expert's feedback** will also be analyzed using **qualitative thematic analysis**, following the same steps:
    - **Familiarization**: Reading and understanding the expert's comments.
    - **Coding**: Capturing key points such as system accuracy, reliability, usability, and educational value.
    - **Categorization**: Grouping similar codes into broader categories.
    - **Theme Identification**: Identifying themes including strengths, limitations, and recommendations for improvement.

This approach ensures that the **expert evaluation complements student feedback**, providing a richer and more comprehensive understanding of both **user experience** and **system quality**.

3.6 ETHICAL CONSIDERATIONS

Ethical considerations are central to this study, which seeks to enhance students' educational experiences through technology. To ensure that the research is conducted responsibly and with integrity, the following measures will be observed:

Confidentiality: The privacy of all participants will be strictly protected. Personal and identifying information will be anonymized during data collection and analysis to prevent results from being traced back to individual students. All data will be securely stored and accessible only to the research team. This approach is consistent with the principles of professional responsibility outlined in the IEEE Code of Ethics.

Data Protection: The study will comply with the Data Privacy Act of 2012 (Republic Act No. 10173) of the Philippines, ensuring that students' data is handled securely and ethically. Collected data will be used solely for research purposes. Participants will be informed of their rights, including the ability to access, correct, or request the deletion of their data at any point during the study.

Informed Consent: Participation in the study will be voluntary, and no student will be required to take part without their explicit agreement. Before participating, students will be fully informed about the objectives, procedures, potential risks, and benefits of the study. Only those who provide written informed consent will be included. Participants will also be reminded that they may withdraw from the study at any time without penalty or negative consequences.

3.7 RAG DEVELOPMENT

3.7.1 Overview of the RAG System

This section introduces the overall architecture of the RAG system. The proposed RAG System Architecture is shown in Figure 4. It is divided into two main tasks: the Document Ingestion Flow and the Query Flow, reflecting the distinct processes of ingesting documents and performing retrieval and generation based on the user's query.

The Document Ingestion Flow highlights the essential process of feeding raw documents into the RAG system. The document contents are first extracted using a combination of techniques, then chunked into multiple sub-documents. The metadata and content are stored in a dedicated content storage, while each chunk is also processed through an embedding model to generate vector representations needed for retrieval. Finally, these vectors are stored in a database to enable efficient retrieval.

The Query Flow illustrates the retrieval process. The user's query is first embedded into a vector representation to support dense retrieval. Before retrieval occurs, however, the query is passed through a classifier model to enable adaptive retrieval based on the complexity of the user's query. After both dense and sparse retrieval are performed, the retrieved documents are fused—this hybrid retrieval approach combines the strengths of two distinct retrieval techniques. A reranker, which is a cross-encoder model, then reorders these documents based on semantic

similarity. The top-ranked documents are passed to the Large Language Model (LLM) as contextual input, where a detailed prompt is assembled. This ensures that the LLM's generated output remains accurate, coherent, and grounded in the content of the retrieved document.

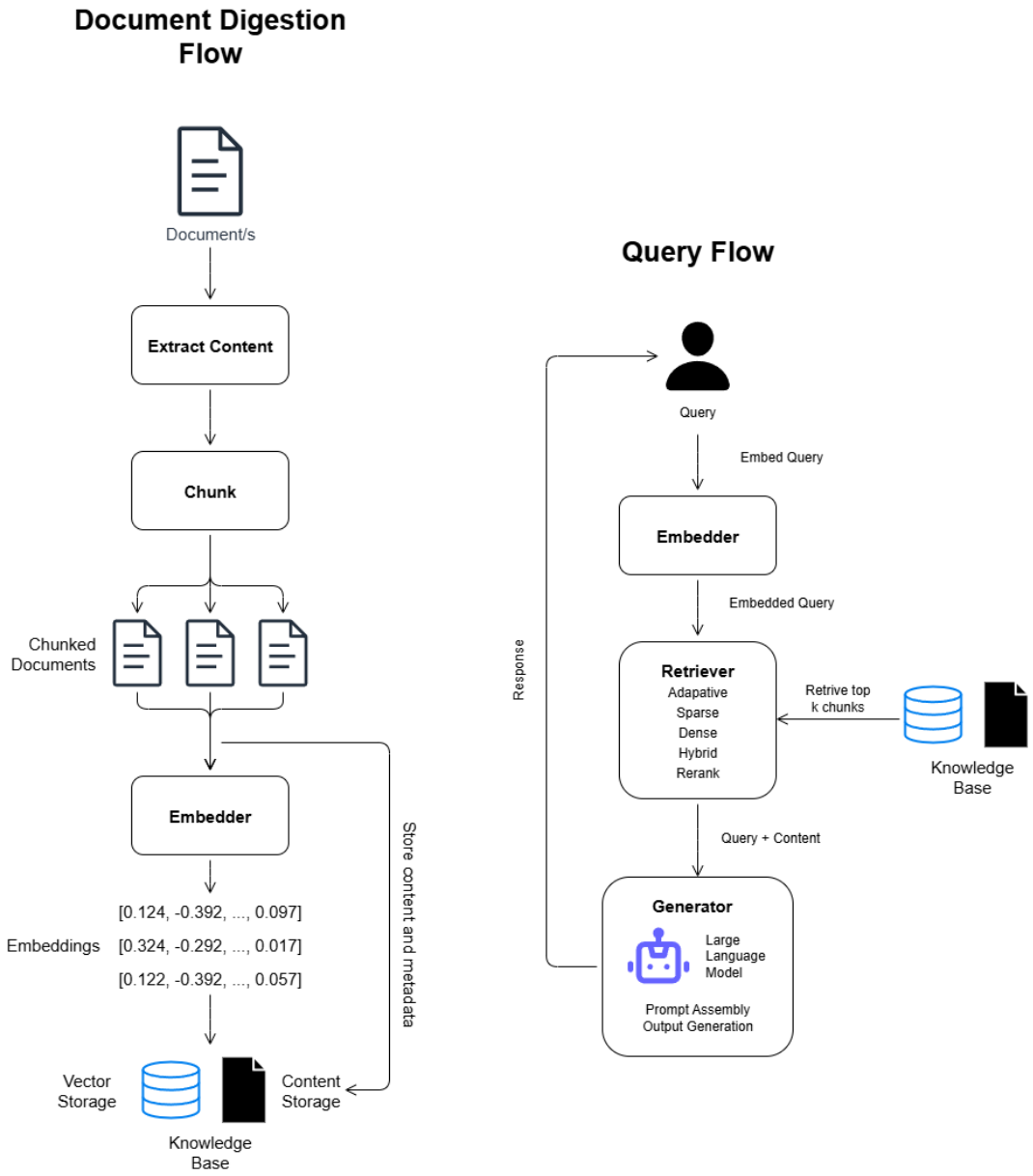

Figure 4. Overview of the RAG System

3.7.2 Data Collection

The primary objective of the data collection phase is to gather comprehensive and high-quality lecture note documents that will serve as the foundation for both the knowledge base and the evaluation of the RAG system. Unlike traditional models, the evaluation of a RAG system is not the same as evaluating a standard LLM because it

involves both the retrieval of relevant documents and the generation of accurate responses. Therefore, this data is critical in assessing the system's ability to retrieve relevant information, generate accurate and complete answers, and maintain overall efficiency. The collected data will be used to evaluate the system on its retrieval and generation quality.

To begin, we will focus on gathering open-source computer science lecture notes, which will form the knowledge base for the system. These lecture notes will be sourced from publicly available documents in PDF and DOCX formats. A total of 30 lecture notes will be collected, covering a wide range of fundamental topics in computer science such as algorithms, data structures, programming languages, and software engineering principles. While most of these documents will contain textual information, some may include images like diagrams. These documents will provide the core knowledge from which the RAG system will retrieve information to answer queries.

Alongside the knowledge base, a human construcsted set of 10 sample queries per lecture notes gathered will be defined to evaluate how well the system retrieves and generates responses. These queries will simulate real-world questions a user might ask in the context of computer science, ranging from simple queries like "What is a sorting algorithm?" to more complex ones requiring detailed answers, such as "Explain the concept of recursion and provide examples." These queries will test the retriever's ability to identify relevant documents and the generator's ability to provide accurate and informative answers [70].

Once the queries are gathered, we will create a query dataset that will serve as the foundation for the entire evaluation process. Each query will be paired with corresponding documents from the knowledge base that are relevant to that query. A column will be added to the dataset indicating whether each document is relevant or non-relevant for the specific query. This data set will be used to assess the system's retrieval quality (i.e., how well the system identifies relevant documents) and its ability to generate accurate answers based on the retrieved documents. The dataset will look like:

Table 2. Query and Responses

Query ID	Query	Document Title	Answer
1	What is sorting algorithm?	Introduction to Algorithms	A sorting algorithm is a method for

Query ID	Query	Document Title	Answer
			arranging elements in a list or array in order.
1	What is sorting algorithm?	Data Structures and Algorithms	A sorting algorithm is a method for arranging elements in a list or array in order.
1	What is sorting algorithm?	Common Algorithms	A sorting algorithm is a way to order elements in ascending or descending order.

This well-structured data collection process will provide a solid foundation for evaluating the system's performance and ensuring that the RAG system developed in this study delivers accurate, comprehensive, and efficient responses.

3.7.3 Ingestion Pipeline

The Ingestion Pipeline handles the ingestion of the documents to be used for the Retrieval of the documents. The documents content is extracted, chunked, and finally embedded to then be stored in a Vector Database. This section explains each process thoroughly on efficient transformation of the Raw documents.

3.7.3.1 File Extraction

The contents of the documents are extracted, wherein the documents like PDF or DOCX documents are turned into text representation. DOCX are converted into PDF then The extraction is performed using a tool called Pdfplumber, which is preferred because it provides more accurate parsing of text compared to many alternatives. Unlike simple PDF-to-text converters. Pdfplumber preseves the logical flow of the document, including columns and spacing. Extract structured elements such as tables headers, and footers, with higher accuracy, and handle complex layouts better, reducing data loss or misalignment. This makes it particularly reliable when working with PDF's that cotain both plain text and structured content [71].

3.7.3.2 Optical Character Recognition

Documents containing images or are scanned that may contain valuable information are extracted using Optical Character Recognition (OCR), a technology that allows computers to recognize and extract text from images or scanned images. We used the OCR library Pytesseract which is free and easily accessible. Our system incorporates an OCR step when the extraction pipeline encounters images within the documents integrating. Through OCR, these details were extracted and made available for use unlike relying solely on the extraction using Pdfplumber which only works on textual data. This process expanded the overall coverage of information and provided a more complete picture of the documents being processed. OCR strengthened the foundation of our system by ensuring that all relevant textual information whether originally stored as text or within images was captured in a consistent format. This comprehensive approach reduced the risk of missing important knowledge and improved the overall quality of the system's resources [72].

3.7.3.3 Chunking

Extracted documents usually are too long, which might cause the retriever to fetch whole corpora which is unnecessary and intensive, to handle this, a chunking strategy is applied. Chunking is the process of dividing long documents into smaller segments, commonly referred to as *chunks*. Each chunk typically contains only a few hundred words, allowing the retrieval system to search more precisely and return only the most relevant segments in response to a query. A chunking strategy is crucial to keep relevant sections together and ensure it doesn't cut mid sentences. Traditional chunking approaches use fixed counts of either word, sentence, or paragraph to determine the length of chunks. While this ensures that the text is manageable for retrieval and processing, it often produces arbitrary cuts that fail to align with the semantic flow of the document. For example, a definition may be split across two chunks, or an example may be separated from the concept it explains. Such breaks reduce retrieval accuracy and increase the likelihood of fragmented or incomplete responses.

A strategy that is gaining popularity is semantic chunking wherein instead of cutting in fixed lengths divides text into coherent units of meaning, ensuring that each chunk contains a logically consistent idea. Instead of relying on static

boundaries, semantic chunking leverages embeddings which are mathematical representations of sentences or paragraphs to measure semantic similarity. The process begins by splitting a document into sentences. Each sentence is then transformed into an embedding vector using a pre-trained sentence transformer all-MiniLM-L6-v2, a lightweight yet powerful transformer-based model that achieves performance comparable to larger models (such as BERT or RoBERTa) while requiring significantly fewer computational resources. The v6 variant is a fine-tuned version of MiniLM, optimized specifically for semantic textual similarity, clustering, and retrieval tasks.

Consecutive embeddings are compared using similarity scores. When the similarity between sentences drops below a threshold, it is treated as a natural boundary, and a new chunk is created. This ensures that only semantically related sentences are grouped together. Additionally, a sliding window with overlapping is sentences from the end of one chunk are repeated at the beginning of the next. This prevents loss of context at boundaries and improves the model's ability to maintain continuity. A maximum token length constraint (e.g., 300–800 tokens) is also enforced to keep chunks within the processing limits of large language models.

3.7.3.4 Embedding

After documents are divided into chunks, each chunk must be converted into a format that allows the retrieval system to efficiently compare their meanings. This is accomplished through embeddings. An embedding is a numerical representation of text in a high-dimensional space, where the distance between two vectors reflects their semantic similarity.

In this study, the Sentence Transformer all-MiniLM-L6-v2 was used to generate embedding. It is a lightweight yet powerful transformer-based model that achieves performance comparable to larger models (such as BERT or RoBERTa) while requiring significantly fewer computational resources. The "v6" variant is a fine-tuned version of MiniLM, optimized specifically for semantic textual similarity, clustering, and retrieval tasks. The embedding steps are:

1. Input Preparation – Each chunk of text, typically consisting of several sentences, is passed into the embedding model.

2. Transformation – The model converts the input into a fixed-length vector of floating-point values (e.g., 384 or 768 dimensions). This vector numerically encodes semantic properties of the text.

3. Semantic Space Mapping – The generated vectors are situated within a high-dimensional semantic space. In this space, chunks that discuss related concepts (e.g., “plants produce food” and “photosynthesis”) are positioned close to each other, even if they use different wording

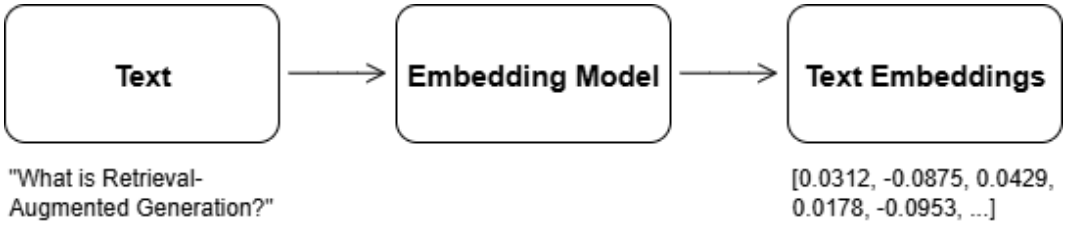

Figure 5. Text Embedding

3.7.3.5 Vector Store

Embedded text chunks are stored in a vector database, and for this system we use FAISS (Facebook AI Similarity Search). FAISS is an open-source library developed by Facebook AI Research that is optimized for efficient similarity search and clustering of dense vectors. It is particularly well-suited for large-scale information retrieval tasks, as it can handle millions of embeddings while supporting both exact nearest neighbor search and approximate methods for faster lookups.

In this study, FAISS serves as the core vector index, enabling the system to quickly retrieve the most semantically relevant chunks given a query embedding. Its ability to perform high-dimensional similarity search (using metrics such as cosine similarity and L2 distance) ensures that the retrieval process is both accurate and efficient. Applied in this way, FAISS allows the system to provide timely, context-aware responses by matching user queries with the most relevant information stored in the database.

3.7.4 Query Pipeline

3.7.4.1 Query Embedding

Query embedding is the process of converting a user’s text query into a numerical vector representation that captures its semantic meaning. This allows the

system to compare the query efficiently with document embeddings in the database. By representing both queries and documents in the same vector space, the system can retrieve the most relevant information based on similarity rather than exact keyword matches.

3.7.4.2 Retrieval

Retrieval is essential for obtaining relevant information in response to a user's query. Before retrieval begins, the query is classified based on its complexity (e.g., easy, medium, or hard). This adaptive approach determines the number of documents to retrieve and the parameters of each retrieval system. The system integrates both sparse and dense retrieval methods, combining them into a hybrid retrieval approach to maximize accuracy and coverage. Once candidate results are gathered, a reranking process prioritizes passages according to their true relevance to the query. Finally, the top-ranked passages are packaged as context and provided to the language model, enabling them to generate factually grounded and coherent answers.

3.7.4.2.1 Adaptive Retrieval

Adaptive retrieval is a smart strategy that makes the retrieval process in RAG systems more efficient by tailoring the search effort to the complexity of the user's query. Instead of using the same retrieval method for every question, this approach first uses a classifier model (like a fine-tuned DistilBERT) to analyze the incoming query and predict its difficulty—classifying it as easy, medium, or hard. Based on this classification, the system dynamically adjusts its retrieval strategy:

- For simple queries, it retrieves fewer documents, saving time and computational resources.
- For complex queries, it can allocate more effort, such as retrieving a larger number of documents or using more advanced search parameters to gather sufficient context.

This method makes the entire RAG system more efficient by ensuring that simple questions are answered quickly without unnecessary work, while complex questions get the thorough investigation they need.

3.7.4.2.2 Sparse Retrieval

Sparse Retrieval is a traditional method in information retrieval where both documents and queries are represented as sparse vectors, meaning most of the entries are zero since only the words that occur in the text receive non-zero values. Sparse Retrieval works by matching exact terms between the query and documents, which is made efficient through an inverted index. An inverted index is a data structure that maps each word in the vocabulary to the list of documents that contain it, allowing the system to quickly find all candidate documents for a query without scanning the entire collection. Once the relevant documents are retrieved, they are scored to determine how well they match the query. An issue with keyword matching is that simply counting word occurrences is not sufficient, since some words appear very frequently (like “the” or “is”) and are less informative, while rarer words provide stronger signals of relevance.

To handle this, the ranking function BM25 is used. BM25 is a Bag-of-Words model, where the presence or frequency of terms is used for matching this improves retrieval by balancing three key factors: the frequency of query terms within a document (term frequency), how rare or informative those terms are across the entire collection (inverse document frequency), and document length normalization so that longer documents do not automatically receive higher scores. In this way, sparse retrieval offers an efficient and interpretable approach to search, relying on keyword overlap enhanced with statistical weighting. We utilized BM25 Ranking for our Sparse Retrieval to make keywords comparison with the query.

BM25 can be expressed as:

$$BM25(D,Q) = \sum_{t \in Q} IDF(t) \cdot \frac{f(t,D) \cdot (k_1 + 1)}{f(t,D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{avgdl}\right)}$$

Equation 2. BM25 Formula

Where:

- **D** = a document
- **Q** = the query
- **t** = a term in the query

- **f(t,d)** = frequency of term *t* in document *d*
- **|D|** = length of document *d* (in words)
- **avgdl** = average document length in the collection
- **k, b** = tuning parameters (commonly *k* ≈ *1.2–2.0*, *b* ≈ *0.75*)

The BM25 IDF (t) formula uses a log function that reduces the impact of very rare terms and stabilizes scoring. The BM25 IDF is calculated as:

$$IDF(t) = \ln\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$$

Equation 3. IDF (t) Formula

Where:

- t = a term
- N = total number of documents in the collection
- n(t) = number of documents containing term t

3.7.4.2.3 Dense Retrieval

Dense vector search, also known as semantic retrieval, has emerged as a foundational technique in modern information retrieval systems. It represents queries and documents as high-dimensional, dense vectors using deep learning models such as BERT or other transformer-based encoders [42]. Unlike sparse retrieval like the BM25 function, which relies on exact word matches, dense retrieval uses semantic similarity, where words, phrases are mapped into a continuous vector space using neural networks this are then retrieved by using Retrieval is then performed by finding documents whose embeddings are close to the query embedding either by using dot product or cosine similarity. The relevant documents are compared in their similarity in a vector space, so exact matches are not relevant in the type of retrieval.

We used Cosine similarity for determining relevant documents within the Dense vector space. Cosine similarity is a metric that measures how similar two vectors are by looking at the angle between them, not their length. By focusing on vector orientation rather than magnitude it is effective in identifying semantic similarity in high-dimensional data, where document length or vector scale should not distort similarity.

In Cosine Similarity For two vectors, A and B:

$$\text{CosineSimilarity}(A, B) = \frac{A \cdot B}{|A|\,|B|}$$

Equation 4. Cosine Similarity Formula

Where:

- **A · B** is the dot product of the vectors,
- ‖**A**‖ and ‖B‖ are the Euclidean norms (magnitudes) of the vectors.

The resulting value ranges from −1 to 1, wherein 1 indicates that the vectors are perfectly aligned, which can be attributed that these two vectors are similar while 0 indicates unrelated and –1 are perfect opposites.

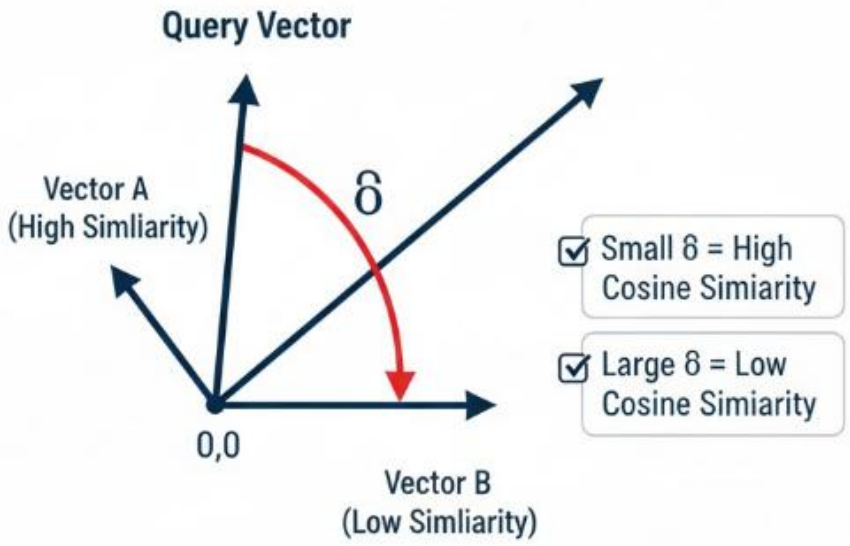

Figure 6. Cosine Similarity

3.7.4.2.4 Hybrid Retrieval

Hybrid Retrieval is an approach that combines both sparse retrieval methods and dense retrieval methods, to take advantage of their complementary strengths. Sparse retrieval excels at finding documents with exact keyword matches, while dense retrieval captures semantic meaning even when different words are used. In hybrid retrieval, we implemented a weighted fusion approach where the relevance scores from both sparse and dense systems are combined to produce a final ranking. For example, a document may receive a BM25 score based on keyword overlap and a cosine similarity score based on embedding similarity, and these scores are merged (through weighted addition, normalization, or other fusion techniques). Empirical studies demonstrate that hybrid retrieval consistently outperforms either method alone across a range of benchmarks [46, 47]. to determine the overall relevance. This way, hybrid retrieval ensures that documents which match the query both literally and semantically are ranked highest, improving accuracy and robustness compared to using either method alone.

The Fused score is expressed as:

$$FusedScore(d) = \alpha \cdot sdense(d) + (1 - \alpha) \cdot sBM25(d),.$$

Equation 5. Fused Score Formula

Where:

- **d** = a candidate document
- **S_dense(d)** = similarity score of document d from the dense retriever (e.g., cosine similarity between embeddings)
- **S_BM25(d**) = similarity score of document d from the sparse retriever (BM25 keyword-based relevance)
- **α** = weighting factor (0 ≤ α ≤ 1) that controls the contribution of dense vs. sparse scores

3.7.4.2.5 Reranker

After retrieving candidate documents, we apply a reranking stage using a pretrained cross-encoder model named BAAI/bge-reranker-base base a cross-encoder reranker model developed by the Beijing Academy of Artificial

Intelligence (BAAI). It belongs to the BGE (BAAI General Embedding) model family, designed for retrieval and reranking tasks in information retrieval (IR) and Retrieval-Augmented Generation (RAG) pipelines. Unlike bi-encoder used in dense retrievers, that transforms a query to find relevant documents into a vector space that independently encode queries and documents into embeddings for similarity comparison in vector space, a cross-encoder jointly processes each query–document pair and produces a direct relevance score (logit). It then computes a relevance score that indicates how well the passage matches the query. This joint encoding allows the model to capture richer semantic interactions between the query and document text, which improves retrieval precision.

The output of the BAAI/bge-reranker-base cross encoder is unbounded. To make easy to interpret scores we applied a sigmoid transformation to map raw logits into a probabilistic like [0,1] range:

$$s(d \mid q) = \sigma\big(\mathrm{CE}(q,d)\big) = \frac{1}{1 + e^{-\mathrm{CE}(q,d)}}$$

Equation 6. Sigmoid Equation

By running a cross encoder to the query-passage pair, we ensured semantic relevance and precision. We then sent the top 10 documents from the Cross encoder reranked as context to the generation module.

3.7.4.3 Generation

3.7.4.3.1 Model Selection

In this project, the answers are generated using a pretrained LLM called Mistral-Instruct. The Mistral model family is designed to offer high performance while remaining more computationally efficient and lighter than some of the larger models, such as GPT-3 or LLaMA-2, making it suitable for applications where resources or response times are constrained. The instruct variant has been fine-tuned on instruction-following datasets, which allows it to interpret user prompts more effectively and produce outputs that closely align with the user's intent, rather than generating generic or unstructured text. By using

Mistral-Instruct, the project leverages a model that already possesses broad general knowledge and strong language generation capabilities, while avoiding the need to train a model from scratch. This ensures that the responses generated are both contextually relevant and accurate, providing an efficient and reliable solution for the task at hand

3.7.4.3.2 Prompt Assembly

Prompt assembly refers to the careful construction of the input provided to the LLM to ensure accurate, contextually grounded, and instruction-aligned responses. The prompt combines the user's query, retrieved context chunks, and explicit instructions to guide the model's behavior as seen in Figure 7. The system prompt instructs the model to use only the information present in the context, avoid inventing facts or citations, and respond with a standard message if the answer cannot be found. Few-shot examples are included to illustrate the expected format and citation style. This structured assembly minimizes hallucinations, enforces factual accuracy, and ensures that the model produces concise, academically appropriate answers.

```
SYSTEM: You are an academic assistant. Use ONLY the information found in the provided context chunks.
Do NOT invent facts or citations. If the answer cannot be produced from the context, reply exactly: "I don't know based on the given context."

CITATION RULES:
- For each factual statement, add an inline citation in this format: [source, page], e.g., [intro, 8].
- Only cite sources present in the context. Do not invent authors, years, or page numbers.

FEW-SHOT EXAMPLES:
Context:
[1] Source: Intro, Page: 8
Content: A sorting algorithm is a method used to rearrange a list of elements into a particular order. Common examples include quicksort, mergesort, and bubble sort.

Question: What is a sorting algorithm?
Desired Output:
A sorting algorithm is a method to rearrange a list of elements into a particular order. Common examples include quicksort, mergesort, and bubble sort [intro, 8].

-----
NOW THE QUERY AND CONTEXT:
Question:
{query}

Context:
{context}

Return the answer in a concise, readable format with inline citations like [source, page]. Do NOT return JSON or extra commentary.
```

Figure 7. Prompt Template

3.7.4.3.3 Output Generation

A permutation notation is a way to represent a function from {1, 2, ... , n} to {1, 2, ... , n}, where the range is the whole set {1, 2, ... , n}. It can be represented by listing the values in order [Probability.pdf, 9]. For example, if n = 3 and a permutation σ maps 1 to 3, 2 to 2, and 3 to 1, it is represented as {3, 2, 1}.

Figure 8. Sample Output

The Large Language Model (LLM) processes the user's query together with the retrieved contextual information and predefined rules or instructions. These combined inputs guide the model in generating a coherent, relevant, and goal-oriented response. As illustrated in Figure 8, this stage represents the reasoning and generation phase of the system, where the LLM synthesizes the provided context and rules to produce the desired output.

3.7.5 Evaluation

The evaluation of the RAG system will be grounded in the data collected during the data collection phase. The primary focus will be on both the retrieval and generation quality. The retrieval quality will be evaluated by recall and precision by the generation quality will be evaluated by faithfulness, accuracy and completeness which all will be explained.

3.7.5.1 Retrieval Quality Evaluation

In the retrieval phase, the system will be tested on its ability to identify the most relevant documents in response to user queries. Each query in the dataset will be paired with a set of documents, which have been annotated as either relevant or non-relevant. The system's performance will be evaluated by comparing the documents it retrieves against these annotations. This will allow us to assess the precision and recall of the retrieval process. Precision measures how many of the retrieved documents are relevant, while recall evaluates how many of the relevant documents the system successfully retrieved. These two metrics will give us insight into the effectiveness of the retrieval mechanism in finding pertinent information from the knowledge base.

3.7.5.1.1 Recall

Recall is the proportion of relevant documents that were retrieved out of all the relevant documents. It answers the question: *Of all the relevant documents, how many did the system successfully retrieve. It can be expressed as:*

$$Recall = \frac{True\ Positive}{True\ Positives + False\ Negatives}$$

Equation 7. Recall Formula

Where:

- **True Positives (TP)** = Number of relevant documents correctly retrieved.
- **False Negatives (FN)** = Number of relevant documents that were not retrieved.

3.7.5.1.2 Precision

Precision is the proportion of relevant documents retrieved out of all documents retrieved. It answers the question: *Of all the documents retrieved, how many were relevant?*

$$\text{Pr}\,e\,cision\ =\ \frac{True\ Positives}{True\ Positives\ +\ False\ Positives}$$

Equation 8. Precision Formula

Where:

- **True Positives (TP)** = Number of relevant documents correctly retrieved.
- **False Positives (FP)** = Number of irrelevant documents retrieved.

3.7.5.2 Generation Quality Evaluation

The quality of generated responses in the RAG system was evaluated using a combination of automatic metrics and human evaluation to ensure both quantitative and qualitative assessment of performance. This dual approach captures not only the textual similarity with reference answers but also the factual correctness, fluency, and relevance of the outputs.

3.7.5.2.1 Automatic Evaluation

1. BLEU (Bilingual Evaluation Understudy)
    a. Evaluates the precision of n-grams in the generated text relative to reference answers.
    b. Helps identify syntactic and lexical similarity.

2. BERTScore

    a. Uses contextual embeddings from pretrained BERT models to evaluate semantic similarity between generated and reference text.

3.7.5.2.2 Human Evaluation

1. Generate Responses

    a. For each query, the system produces an answer using the retrieved documents.

2. Evaluate Faithfulness

    a. Check if the response remains consistent with the retrieved lecture notes.

    b. Identify any hallucinations, unsupported claims, or contradictions.

3. Evaluate Accuracy

    a. Verify whether the response is factually correct based on the retrieved documents.

    b. Flag incorrect or misleading statements.

4. Evaluate Completeness

    a. Assess whether the response fully addresses all aspects of the query.

    b. Responses that omit key details are considered incomplete.

Table 3. Human Evaluation Metrics

Criterion	2 (High)	1 (Partial)	0 (Low)
Faithfulness	Fully consistent with lecture notes; no hallucinations	Minor unsupported claims, mostly faithful	Major contradictions or hallucinations
Accuracy	All facts correct	Minor factual errors	Largely incorrect or misleading

Completeness	Fully addresses all aspects of the query	Partially covers the query	Incomplete or vague

3.8 SYSTEM PROTOTYPE DEVELOPMENT

Qubo is developed using React.js in the frontend and FastAPI as its backend, along with the Retrieval-Augmented Generation layer. The frontend provides functions such as login, chatting with Qubo, uploading and managing lecture notes. While the backend manages to expose the Api's, handles data flow and security, and connects with the RAG layer. The RAG layer serves as the core intelligence of the platform, combining document retrieval with large language model (LLM) Mistral: Instruct for the generation to deliver accurate, context-grounded answers derived from lecture notes.

3.8.1 System Architecture

Figure 9 illustrates the system architecture and the flow of data within the application. The system is accessed through a User Interface (UI) where users can upload documents and submit queries. These inputs are processed by a backend API that orchestrates the data flow among the core services which constitute the Retrieval-Augmented Generation (RAG) pipeline: the File Service, the Retrieval Service, and the Generation Service. Finally, the Generation Service, which utilizes a Large Language Model (LLM), synthesizes the retrieved information to formulate a response that is delivered back to the user through the UI.

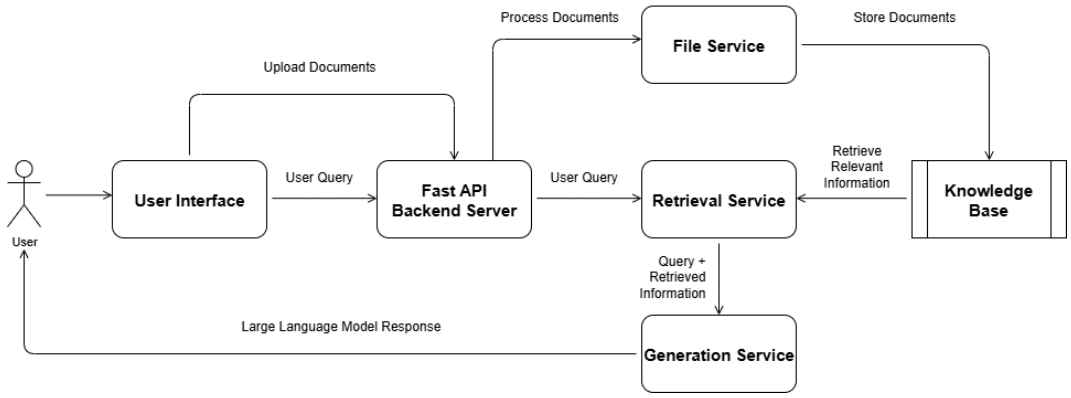

Figure 9. System Architecture

Frontend (User Interface)– The user interface is built using React, for its component-based structure and fast development cycle. React allows us to design a

responsive and dynamic interface where users can submit queries, view retrieved documents and interact with generated answers seamlessly. It also integrates well with backend APIs and supports state management libraries that improve user experience during asynchronous operations. Additionally, React's ecosystem provides flexibility for future enhancements, such as integrating dashboards, visual analytics, or real-time feedback collection.

Backend (API)- The backend of the system is developed using FastAPI, a modern Python web framework designed for building high-performance APIs. It serves as the central orchestration layer that connects the user interface, the RAG layer, and the knowledge stores. FastAPI was chosen because of its asynchronous request handling, automatic data validation, and compatibility with machine learning workflows, which make it efficient for handling multiple queries simultaneously. To run the FastAPI application, the system uses Uvicorn, a lightning-fast ASGI server built for asynchronous Python frameworks. Uvicorn handles incoming HTTP requests, manages connections, and ensures low-latency communication between the client and the backend. Its performance and seamless integration with FastAPI make it a production-ready choice for deploying the system.

Knowledge Base (Storage)- The Knowledge Base is a compilation of databases that maintain structured data such as user profiles, authentication records, and system logs using MySQL lite For unstructured and semantic search, a vector database built with FAISS stores embeddings of documents along with a content storage file that contains document content and metadata, enabling fast and accurate similarity search.

RAG Layer (Model Integration)– The RAG layer serves as the AI logic of the system and consists of three interconnected modules: indexing, retrieval, and generation. The indexing module converts documents into vector embeddings and stores them in FAISS for later access. The retrieval module fetches the most relevant passages from the knowledge store based on user queries. Finally, the generation module, powered by the LLM, synthesizes the final answer by combining the user's query with the retrieved context. This layered approach ensures both efficiency and factual grounding in responses.

3.8.2 Design Methodology

This study utilized Agile software development management as the primary design methodology. Agile was selected over traditional approaches such as Waterfall and Scrum because of its flexibility, iterative nature, and strong emphasis on continuous progress throughout the development lifecycle. Unlike Waterfall, which relies on a sequential and rigid process, Agile allows for incremental development, enabling the team to respond dynamically to changes in requirements, user feedback, or technical challenges. While Scrum is a framework under Agile that focuses on sprints and defined roles, this study adopted the broader Agile principles to maintain flexibility in task management and rapid iteration without being confined to a strict sprint cycle.

3.8.3 TECHNOLOGY STACK

React.js: Selected for building the front end of the system because it is a widely used JavaScript library that allows the creation of interactive, component-based user interfaces. Its virtual DOM and efficient rendering make it suitable for responsive applications where query input and real-time output streaming are important.

FastAPI: FastAPI was chosen for the backend implementation because it is a high-performance Python web framework designed for building APIs. It provides asynchronous request handling, automatic validation, and easy integration with machine learning models. This makes it particularly effective for orchestrating the RAG workflow, handling multiple user queries concurrently, and connecting with both the knowledge stores and AI components.

LangChain: Chosen as the LLM application development framework because it provides a rich API that abstracts much of the complexity involved in building LLM-powered systems. It simplifies prompt management, chaining of retrieval and generation steps, and integration with vector databases, which makes it ideal for implementing a Retrieval-Augmented Generation (RAG) pipeline.

pdfplumber: Used for extracting structured text from PDF documents. It allows parsing of layouts, tables, and text blocks, which ensures that academic lecture notes in PDF format are accurately converted into machine-readable content suitable for indexing and retrieval.

PyTesseract: A Python wrapper for Google's Tesseract-OCR engine. It is used to extract text from image-based documents, scanned lecture notes, and embedded

diagrams within PDFs. This ensures that non-digital text is captured and incorporated into the RAG knowledge base.

FAISS (Facebook AI Similarity Search): FAISS was selected as the vector database because it is optimized for fast similarity search and clustering of high-dimensional embeddings. It can efficiently handle millions of vectors, supporting both exact and approximate nearest neighbor searches. This makes retrieval of semantically relevant document chunks both scalable and efficient.

SQLite: was chosen as a lightweight, version-controlled database for storing user credentials. Its simplicity and minimal setup make it suitable for smaller scale systems.

all-MiniLM-L6-v2: was chosen as the embedding model due to its balance of lightweight architecture and strong performance. It generates high-quality embeddings that capture semantic meaning while being computationally efficient, making it practical for real-time retrieval tasks without requiring large GPU resources.

BAAI/bge-reranker-base: To enhance retrieval precision, the system incorporates the BAAI/bge-reranker-base model as a reranker. While FAISS retrieves a broad set of relevant candidate chunks, the reranker assigns finer-grained relevance scores and reorders results so that the most contextually appropriate passages are prioritized. This ensures higher-quality inputs to the generation step, improving overall system accuracy.

Mistral-Instruct: Mistral-Instruct, a fine-tuned large language model for instruction following, was selected as the generator because it produces coherent, instruction-aligned responses. Its fine-tuning ensures it can follow user queries closely, making the system's answers more accurate, grounded, and useful.

3.8.4 Module Development

3.8.4.1 Frontend Development

The front-end part of the web application was developed using React, which is an open-source library widely used to build fast and dynamic user interfaces. React's component-based architecture allows for efficient updates and rendering The Frontend is the visual interface that students use to interact with the Qubo system, providing access to essential features like authentication, file management, and the core chatbot functionality.

Students are required to **log in or sign up** to gain authenticated access to Qubo's features, such as document uploads and the chatbot as shown Figure 10. When a new student signs up, their secure credentials are saved, and they are immediately issued a web token (a secure digital key) to start using the application. This token is attached to every subsequent request, verifying the student's identity and maintaining a secure session without forcing them to re-enter their login details. This process, secured by encrypted communication (HTTPS), ensures that only verified users can access protected features like document storage and personalized chatbot interactions.

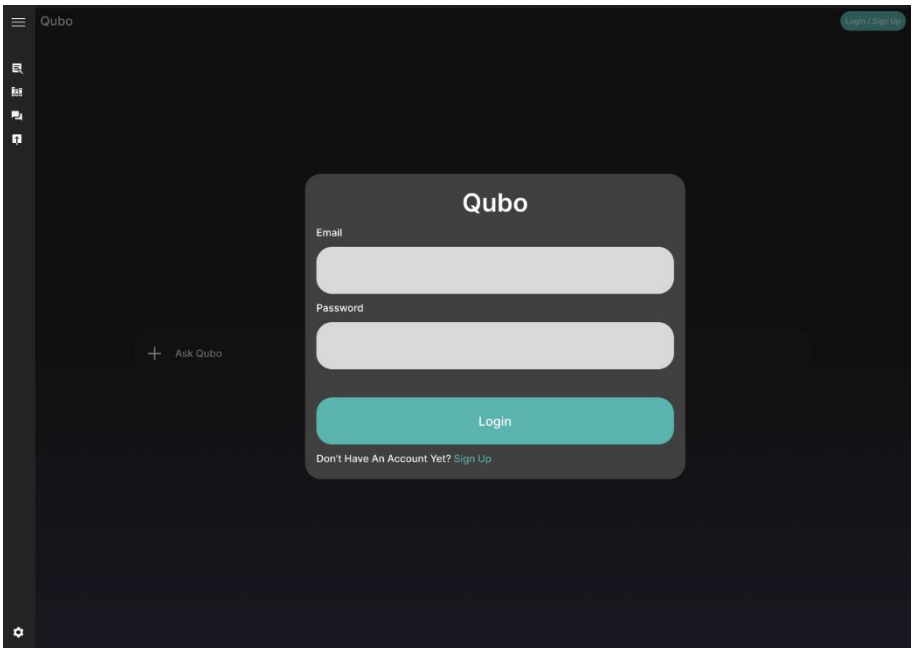

Figure 10. User Login/Sign Up

Figure 11 illustrates Chatbot Interface, the primary screen where students submit their questions in natural language. Users type their queries into a text field and send them to the backend service.

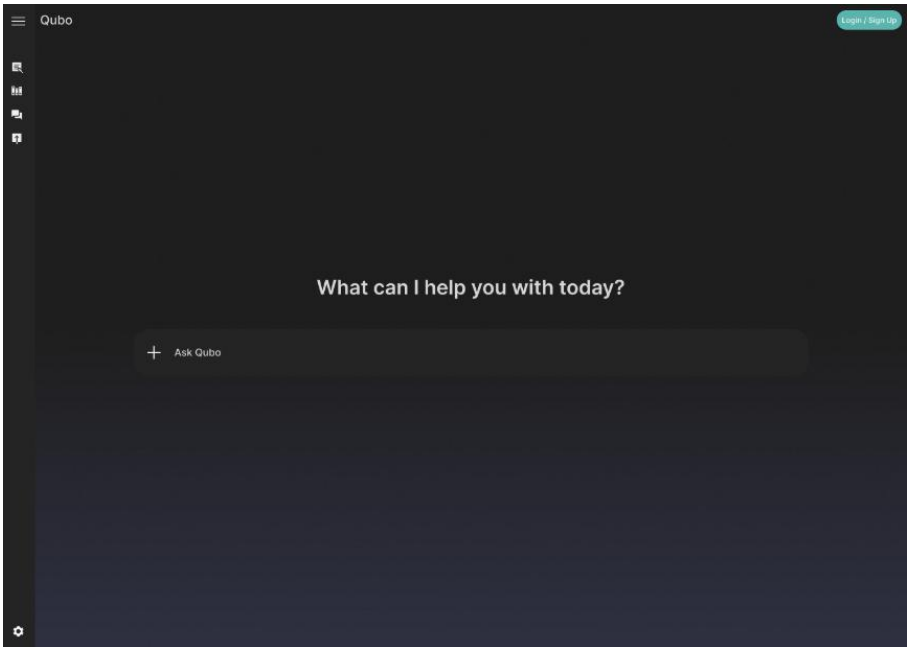

Figure 11. Chatbot Interface

The backend processes the query, performs retrieval and generation operations, and sends an accurate, grounded response back to be displayed instantly on the screen as shown in Figure 12.

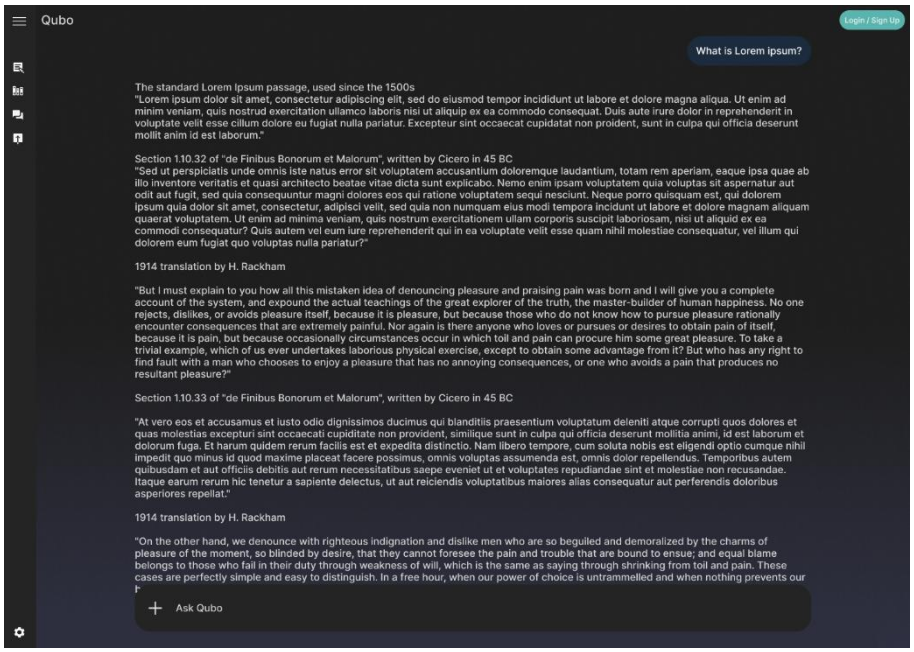

Figure 12. Chatbot Interface - Query and Anwer

A crucial feature of this interface is the inclusion of reference buttons on the generated answers. As show in Figure 13 Upon clicking a reference button, a Side Panel automatically appears, which grounds the answer by showing the

student exactly which part of the source file (the specific lecture notes or document) the information was taken from.

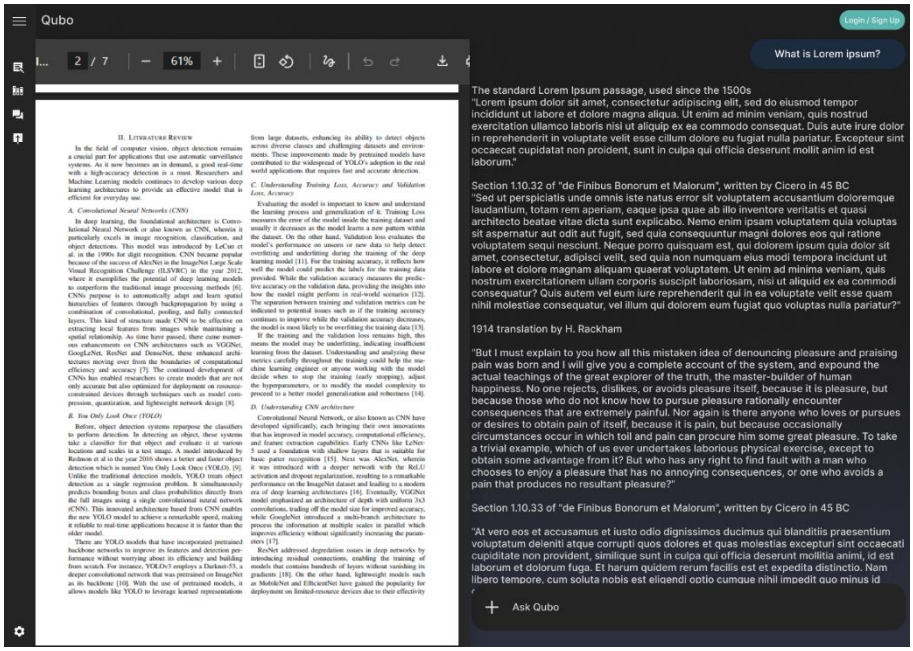

Figure 13. Chatting Page - Reference Panel

Figure 14 illustrates the File Manager (also referred to in the interface as the File) is a secure area available only to logged-in students. This is the central hub for managing the documents that Qubo will use for answering questions. Students can view, upload, and organize their lecture notes and course materials, which support common formats like PDF, DOCX, and plain text. The files uploaded here are sent to the backend to create the knowledge base. This is the core data source for the Retrieval-Augmented Generation (RAG) process, where the system will later select the most relevant passages to answer a student's query.

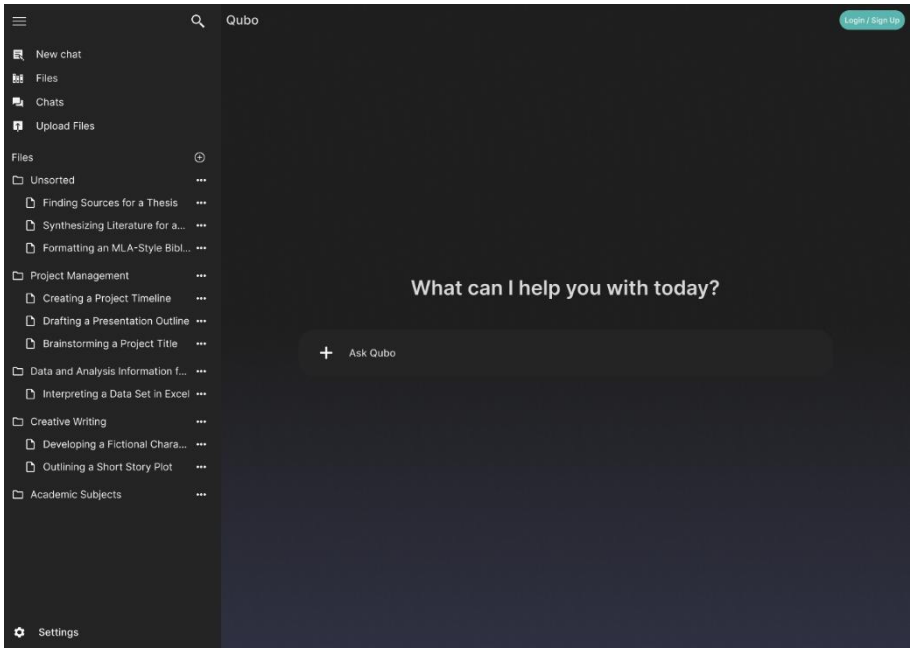

Figure 14. Side Panel - Files

The Side Panel acts as a comprehensive navigation and settings hub, providing students with full control over their Qubo experience. It can be minimized using a triple bar icon, and when minimized, it shows a compact view with an expand button, along with quick links for New Chat, Files, Chats, and Upload Files as shown in Figure 15

The panel is searchable via a Search option to quickly find files or chats. It contains several key sections:

As seen in Figure 14 the settings button is a dedicated area for accessing general application configuration options. The “Upload Files” button provides a quick access point that allows students to easily upload all the necessary course materials they want Qubo to reference. The “New Chat” button enables the user to start a fresh conversation with the AI instantly.

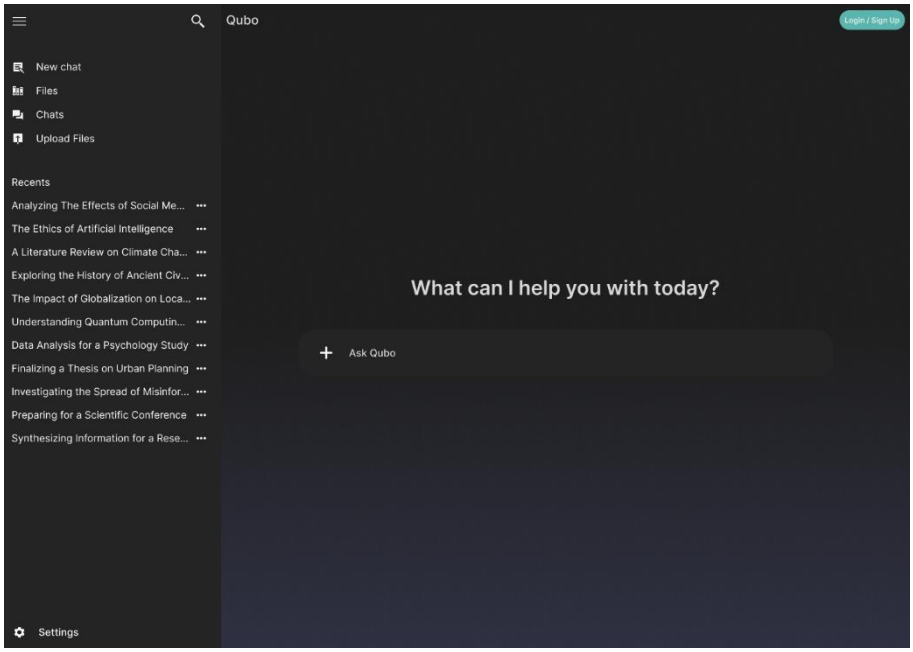

Figure 15. Side Panel

The “Files” library functions as the primary File Manager, displaying all uploaded documents as shown in Figure 16. Files are initially unsorted, but the user has the capability to organize them into folders. For folder management, the user has options to rename the folder and delete the folder. Within the library, for individual files, the user can preview the file (which, upon clicking, splits the chat section to make room for the file preview), download the file, pin the file, rename the file, move to a different folder, and delete the file. Crucially, the File Library also allows the user to select the files they want the chatbot to use as its knowledge source for the current session.

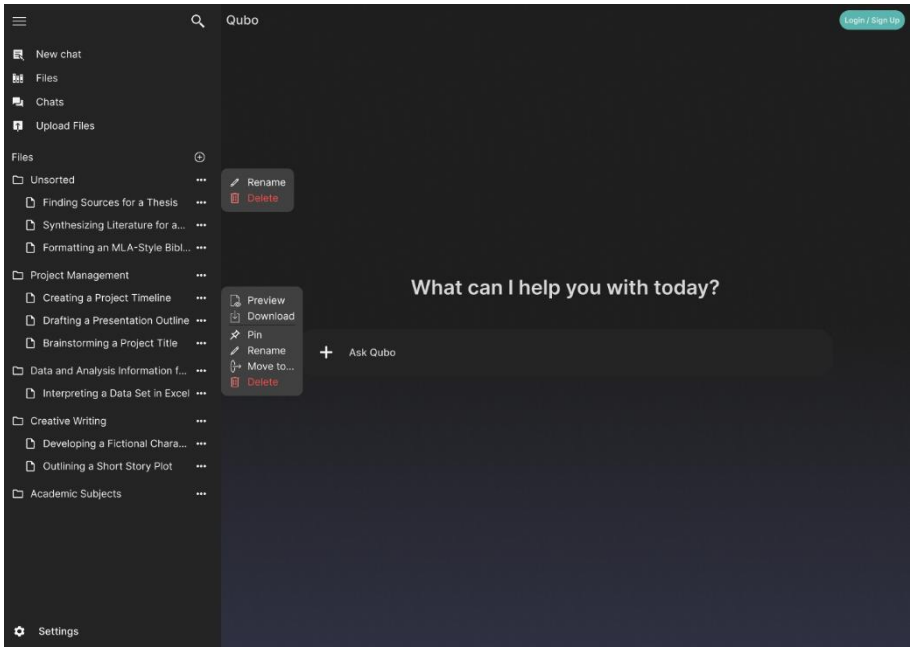

Figure 16. Side Panel - File Options

The Chats section stores the user's recent and existing chat history as show in Figure 17. Students can manage their conversations using options to download the entire conversation, pin meaningful conversations to the top for easy access, rename chats for better organization, or delete them.

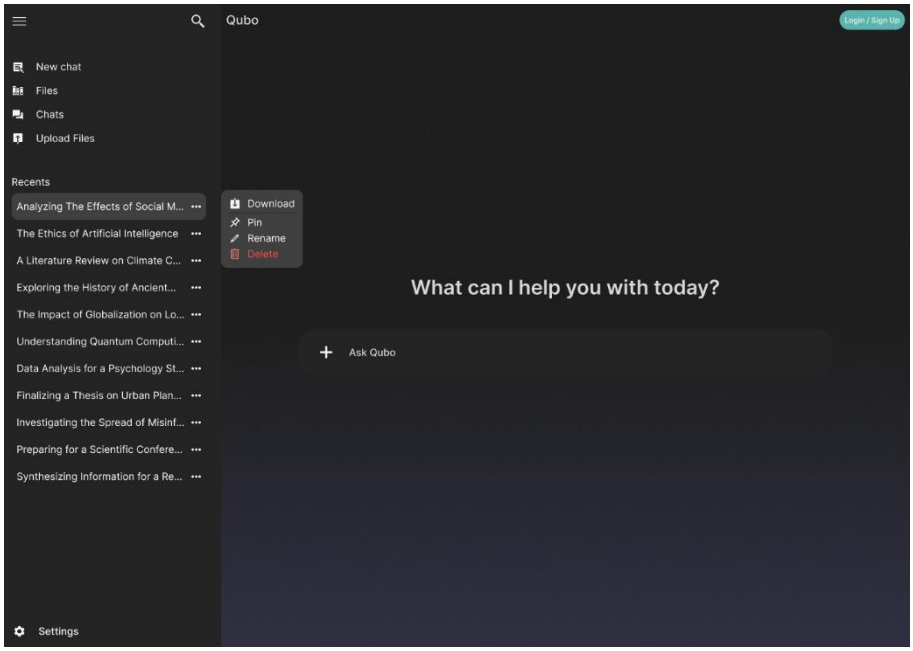

Figure 17. Side Panel - Chats Options

3.8.4.2 Backend Development

The backend of the system is implemented using Fast API, for its speed, scalability, and ease of integration with AI components. It serves as the bridge

between the frontend and the RAG layer, exposing well-defined APIs that allow secure communication and efficient coordination across modules.

RAG Layer: Utilize the Langchain framework to build the base RAG framework.

Database Configuration: SQLite is used for storing user data, configurations, and permissions. SQLite provides a lightweight, serverless solution that fits well with applications that require efficient data storage without the overhead of more complex database systems.

Vector database: The application employs Faiss as its vector database for storing and managing document vectors efficiently.

The exposed APIs encompass the following key functionalities:

Auth API: Handles the user login and sign-up logic, returning the current user details, and managing the json web token(JWT) authentication.

Chat API: This api receives the user queries from the frontend, calls the retrieval-augmented generation services for retrieving relevant documents and generating responses to be sent back to the frontend.

File Upload API: Handles the document uploads within the system by calling the ingestion pipeline to chunk the files, store them in local database along with the metadata, and then use it as embeddings to be stored in the knowledge base

3.8.4.3 RAG Integration

The Retrieval-Augmented Generation (RAG) framework was implemented using LangChain and integrated into the system as a set of service providers. RAG integration is structured into three main services:

1. File Service – This service handles the preprocessing and indexing of documents. It prepares the raw documents by chunking, cleaning, and embedding them, storing the embeddings in the vector database for efficient retrieval.

2. Retrieval Service – This service is responsible for fetching relevant document chunks based on the user's query. It queries the vector database (FAISS) to identify the most semantically relevant content.

3. Generation Service – This service calls the large language model (LLM) and uses the retrieved documents to generate a coherent and contextually accurate response to the user's query. It structures and synthesizes the retrieved information into a usable output.

3.8.4.4 Testing and Refinement

After the system development and integration of the RAG framework, Testing is conducted in the application, table 4 displays the testing matrix, which includes the test cases, expected results, conducted significant application testing to ensure that all functions within the application are performed properly and. The system integration and performance testing were within on the methodologies applied. This chart displays the testing matrix which includes the test cases and expected results and actual outcomes during the testing process.

Table 4. Test Cases

Test Case ID	Feature/ Module	Description	Expected Results	Actual Results
TC01	Login	Check that the user can log in using valid credentials.	Dashboard should appear.	Dashboard loaded successfully.
TC02	Registrati on	Check if a new user can register using valid information.	Account created successfully.	Account created successfully

Test Case ID	Feature/ Module	Description	Expected Results	Actual Results
TC03	File Upload	Ensure that users can upload lecture notes in PDF, DOCX or TXT formats.	File uploads successfully and appears in the list.	File uploaded and displayed correctly.
TC04	Chat Function	Test the chatbot's response when the user asks a question.	Relevant and contextual answers are generated.	Response generated accurately and on time.
TC05	Rag reliability	Confirm that the chatbot retrieves relevant content from the uploaded lecture notes.	The retrieved data should match the query context.	Correct a part was obtained from the knowledge base.
TC06	Respons e Accuracy	Check the actual accuracy with the user generated response.	Answer aligns with source material and is accurate.	Generated answer consistent with uploaded notes.
TC07	Interface Navigatio n	Check the navigation between the chat screen and the file	A smooth transition between pages.	There is no delay and the transitions are smooth.

Test Case ID	Feature/ Module	Description	Expected Results	Actual Results
		management area.		
TC08	Logout Function	Confirm that the user may successfully log out.	User session ends and login screen reappears.	User logged out successfully.
TC09	Error Handling	assure that incorrect uploads (unsupported file types) provide a correct error message.	An unsupported file format message appears.	Error displayed correctly.
TC10	Performance Testing	Measure the average response time during all the queries.	Response Time	The Average response per seconds

3.9 THEORETICAL FRAMEWORK

Figure 18. Theoretical Framework

The design and development of our Retrieval-Augmented Generation chatbot is guided by an integration of educational philosophy and advanced computer science. The project takes root in Constructivist Learning Theory, Natural Language Processing, Information Retrieval Theory, Representation Learning Theory, and Deep Learning.

At its core, the project is rooted in the constructivist learning theory, which states that students learn most effectively by actively building their own understanding. Our chatbot embodies this principle by acting as an adaptive learning partner, helping to manage cognitive load and guiding students through lecture notes in a step-by-step manner to foster active learning. This educational goal is brought to life through a foundation in Natural Language Processing, which governs how the system understands text, leverages large language models, and uses prompt engineering to elicit desired responses. To find the most relevant information for a student's query, the chatbot employs Information Retrieval theory, utilizing a hybrid technique that

merges sparse and dense retrieval for superior accuracy. This process is made possible by Representation Learning, which transforms text into numerical embeddings a computer can search through using cosine similarity within a specialized Faiss database. The entire technological framework is powered by Deep Learning, by utilizing advanced algorithms and techniques, the capabilities required to keep up with modern standards in computing to create assistive digital tools.

3.10 CONCEPTUAL FRAMEWORK

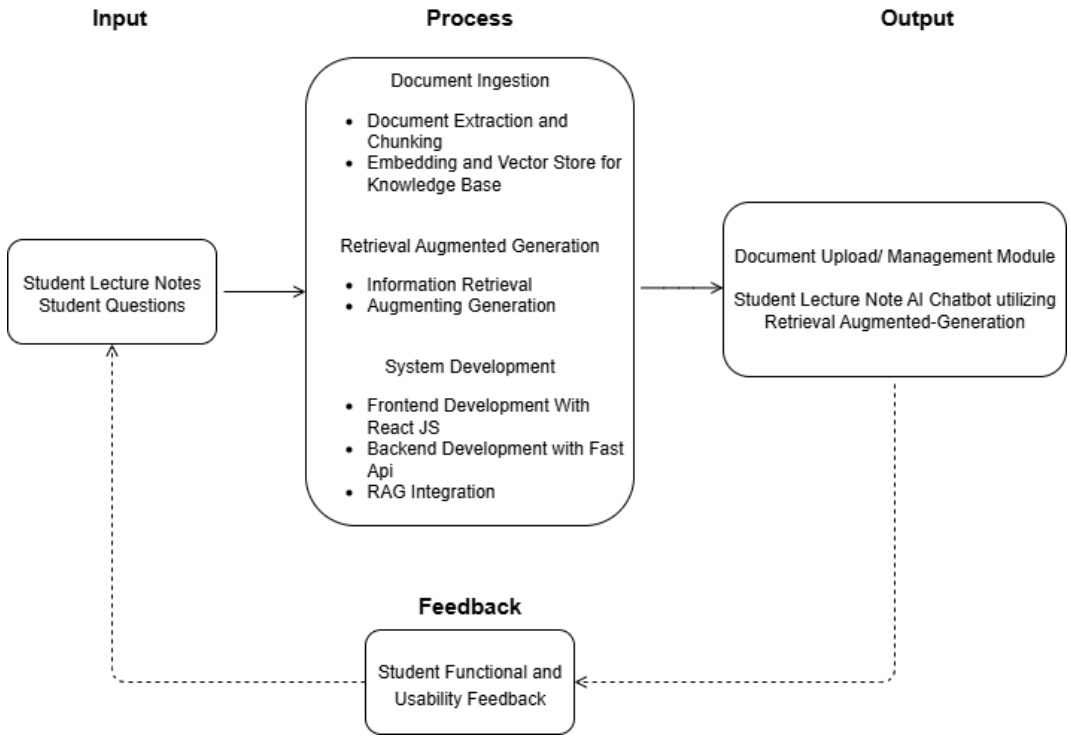

Figure 19. IPO Model

Figure 19 illustrates the study's Conceptual Framework, which uses an Input-Process-Output (IPO) model to depict the system's logical flow. The **input** stage requires student lecture notes to create a knowledge base for information retrieval. The **process** begins when a student asks a question, causing relevant information to be retrieved and augmented into the generation step. A Large Language Model then synthesizes this information to create **an output** grounded in the retrieved documents. This response is then submitted back to the user interface to help with the students' questions. Finally, feedback is collected to gather qualitative and quantitative insights for the continuous enhancement of the chatbot system. The system is built using React.js for the frontend and FastAPI for the backend, along with the integration of RAG model services.

REFERENCES

[1] I. Bouchrika, “Overcoming information overload in higher education: The power of document summarization,” *Research.com*, Sep. 16, 2025. [Online]. Available: https://research.com/education/overcoming-information-overload-in-higher-education

[2] O. M. Adekoya and C. Akune, “Information overload and its effects on academic performance of students,” *Global Review of Library and Information Science (GRELIS)*, vol. 19, no. 2, 2023. [Online]. Available: https://www.researchgate.net/publication/372310697_INFORMATION_OVERLOAD_AND_ITS_EFFECTS_ON_ACADEMIC_PERFORMANCE_OF_STUDENTS

[3] E. Novak, “Factors that impact student frustration in digital learning environments,” *Education and Information Technologies*, vol. 28, no. 3, pp. 923–934, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666557323000319

[4] I. Koenig-Workman, “Cognitive Load Theory,” *The Decision Lab*, Dec. 04, 2024. [Online]. Available: https://thedecisionlab.com/reference-guide/psychology/cognitive-load-theory

[5] I. Abeysekera, E. Sunga, A. Gonzales, and R. David, “The effect of cognitive load on learning memory of online learning accounting students in the Philippines,” *Sustainability*, vol. 16, no. 4, p. 1686, 2024. [Online]. Available: https://doi.org/10.3390/su16041686

[6] A. Mowreader, “65 percent of students use Gen AI chat bot weekly,” *Inside Higher Ed*, Jun. 11, 2025. [Online]. Available: https://www.insidehighered.com/news/student-success/academic-life/2025/06/11/65-percent-students-use-gen-ai-chat-bot-weekly

[7] A. Mowreader, “Students who lack academic confidence more likely to use generative AI for school,” *Inside Higher Ed*, Sep. 30, 2025. [Online]. Available: https://www.insidehighered.com/news/student-success/academic-life/2025/09/30/students-who-lack-academic-confidence-more-likely-use

[8] K. I. Roumeliotis and N. D. Tselikas, “ChatGPT and Open-AI models: A preliminary review,” *Future Internet*, vol. 15, no. 6, p. 192, May 2023. [Online]. Available: https://doi.org/10.3390/fi15060192

[9] A. Deschenes and M. McMahon, “A survey on student use of generative AI chatbots for academic research,” *Evidence Based Library and Information Practice*, vol. 19, no. 2, Art. 2, 2024. [Online]. Available: https://doi.org/10.18438/eblip30512

[10] A. J. Oche, A. G. Folashade, T. Ghosal, and A. Biswas, “A Systematic Review of Key Retrieval-Augmented Generation (RAG) Systems: progress, gaps, and future directions,” *arXiv.org*, Jul. 25, 2025. [Online]. Available: https://arxiv.org/abs/2507.18910

[11] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, Nov. 2022. [Online]. Available: https://doi.org/10.1145/3571730

[12] Ş. Gökçearslan, C. Tosun, and Z. G. Erdemir, “Benefits, challenges, and methods of artificial intelligence (AI) chatbots in education: A systematic literature review,” *International Journal of Technology in Education*, vol. 7, no. 1, pp. 19–39, 2024. [Online]. Available: https://doi.org/10.46328/ijte.600

[13] J. Swacha and M. Gracel, “Retrieval-Augmented Generation (RAG) chatbots for Education: A survey of applications,” *Applied Sciences*, vol. 15, no. 8, p. 4234, Apr. 2025. [Online]. Available: https://doi.org/10.3390/app15084234

[14] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS 2020)*. Curran Associates, Inc., 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[15] M. M. Kaggwa et al., “Prevalence of burnout among university students in low- and middle-income countries: A systematic review and meta-analysis,” *PLoS ONE*, vol. 16, no. 8, p. e0256402, Aug. 2021. [Online]. Available: https://doi.org/10.1371/journal.pone.0256402

[16] J. Z. Schnieders, “Disruptions and gains: Students’ reflections on the effects of the pandemic,” *ACT Research*, Feb. 2023. [Online]. Available: https://www.act.org/content/dam/act/unsecured/documents/R2295-Disruptions-Gains-Effects-of-Pandemic-02-2023.pdf

[17] S. Wang, F. Wang, Z. Zhu, J. Wang, T. Tran, and Z. Du, “Artificial intelligence in education: A systematic literature review,” *Expert Systems With Applications*, vol. 252, p. 124167, May 2024. [Online]. Available: https://doi.org/10.1016/j.eswa.2024.124167

[18] A. M. Al-Zahrani, “Unveiling the shadows: Beyond the hype of AI in education,” *Heliyon*, vol. 10, no. 9, p. e30696, May 2024. [Online]. Available: https://doi.org/10.1016/j.heliyon.2024.e30696

[19] A. K. Abdallah, A. M. Alkaabi, D. A. F. Mehiar, and Z. A. J. Aradat, “Chatbots in classrooms,” in *Advances in Educational Technologies and Instructional Design*, 2024, pp. 166–181. [Online]. Available: https://doi.org/10.4018/979-8-3693-0880-6.ch012

[20] G. Lang and T. Gürpinar, “AI-Powered Learning Support: A Study of Retrieval-Augmented Generation (RAG) Chatbot Effectiveness in an Online Course,” *Information Systems Education Journal*, vol. 23, no. 2, pp. 4–13, 2025. [Online]. Available: https://eric.ed.gov/?id=EJ1467995

[21] H. Soliman, H. Kotte, M. Kravcik, and N. Duong-Trung, “Retrieval-Augmented Chatbots for scalable educational support in higher education,” *ResearchGate*, Mar. 2025. [Online]. Available: https://www.researchgate.net/publication/389582355_Retrieval Augmented_Chatbots_for_Scalable_Educational_Support_in_Higher_Education

[22]N. Kumar, “70 AI in Education Statistics & Trends (2025),” *DemandSage*, 11 August 2025. [Online]. Available: https://www.demandsage.com/ai-in-education-statistics/. [Accessed: 08 October 2025].

[23] BrowserCat Team, “ChatGPT’s Rapid Growth and Usage Among Students (2020-2025),” *BrowserCat*, 18 February 2025. [Online]. Available: https://www.browsercat.com/post/chatgpt-usage-statistics-2020-2025. [Accessed: 08 October 2025].

[24] S. S. Rahman et al., “Hallucination to Truth: A review of Fact-Checking and Factuality Evaluation in large language models,” *arXiv.org*, Aug. 05, 2025. [Online]. Available: https://arxiv.org/abs/2508.03860

[25] M. Serey, J. S. Taboada, and M. M. R. Fernandez, “Global Perspectives on Education: A Cross-Cultural Comparisons in the Philippines and Cambodia,” *Journal of Learning and Educational Policy*, no. 44, pp. 46–56, Jul. 2024. [Online]. Available: https://doi.org/10.55529/jlep.44.46.56

[26] M. A. Pérez-Juárez, D. González-Ortega, and J. M. Aguiar-Pérez, “Digital Distractions from the Point of View of Higher Education Students,” *Sustainability*, vol.

15, no. 7, Art. no. 6044, 2023. [Online]. Available: https://www.mdpi.com/2071-1050/15/7/6044. [Accessed: 08-Oct-2025].

[27] K.-A. Thornby, G. A. Brazeau, and A. M. H. Chen, “Reducing student workload through curricular efficiency,” *American Journal of Pharmaceutical Education*, vol. 87, no. 8, p. 100015, Apr. 2023. [Online]. Available: https://doi.org/10.1016/j.ajpe.2022.12.002

[28] S. Sharma and D. K. Saxena, “Understanding the cognitive constituents of E-Learning,” in *Advances in Educational Technologies and Instructional Design*, 2024, pp. 277–312. [Online]. Available: https://doi.org/10.4018/979-8-3693-4407-1.ch010

[29] N. Zdravkov, “Shaping Minds, Shaping Machines: The Quiet Revolution of AI in UNESCO’s Educational Dream – Student Theses Faculty of Behavioural and Social Sciences (public).” [Online]. Available: https://gmwpublic.studenttheses.ub.rug.nl/4800/

[30] A. P. Murdan and R. Halkhoree, “Integration of Artificial Intelligence for educational excellence and innovation in higher education institutions,” Jun. 2024, pp. 1–6. [Online]. Available: https://doi.org/10.1109/sesai61023.2024.10599402

[31] C. Merino-Campos, “The Impact of Artificial intelligence on Personalized Learning in Higher Education: A Systematic review,” *Trends in Higher Education*, vol. 4, no. 2, p. 17, Mar. 2025. [Online]. Available: https://doi.org/10.3390/higheredu4020017

[32] R. Sajja, Y. Sermet, D. Cwiertny, and I. Demir, “Integrating AI and learning analytics for Data-Driven pedagogical decisions and personalized interventions in education,” *Technology Knowledge and Learning*, Aug. 2025. [Online]. Available: https://doi.org/10.1007/s10758-025-09897-9

[33] C. J. P. Estrellado, “Artificial intelligence in the Philippine educational context: Circumspection and future inquiries,” *ResearchGate*, May 2023. [Online]. Available: https://doi.org/10.29322/IJSRP.13.04.2023.p13704

[34] UNESCO, “Artificial intelligence in education.” [Online]. Available: https://www.unesco.org/en/digital-education/artificial-intelligence

[35] L. Labadze, M. Grigolia, and L. Machaidze, “Role of AI chatbots in education: systematic literature review,” *International Journal of Educational Technology in Higher*

Education, vol. 20, no. 1, Oct. 2023. [Online]. Available: https://doi.org/10.1186/s41239-023-00426-1

[36] J. Swacha and M. Gracel, “Retrieval-augmented generation (RAG) chatbots for education: A survey of applications,” *Applied Sciences*, vol. 15, no. 8, p. 4234, 2025. [Online]. Available: https://doi.org/10.3390/app15084234

[37] S. Eteng-Uket and E. Ezeoguine, “The impact of artificial intelligence chatbots on student learning: A quasi-experimental analysis of learning outcome and engagement,” *Journal of Educators Online*, 2022. [Online]. Available: https://files.eric.ed.gov/fulltext/EJ1470567.pdf

[38] IBM, “What is information retrieval?” *IBM Think*, 2024. [Online]. Available: https://www.ibm.com/think/topics/information-retrieval

[39] GeeksforGeeks, “What is information retrieval?” 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/what-is-information-retrieval/

[40] F. Rosa, R. Rodrigues, R. Lotufo, and R. Nogueira, “Yes, BM25 is a strong baseline for legal case retrieval,” *arXiv preprint arXiv:2105.05686*, 2021. [Online]. Available: https://arxiv.org/abs/2105.05686

[41] Ashikuzzaman, “What is information retrieval and why does it matter?” *Lise Edu Network*, 2025. [Online]. Available: https://www.lisedunetwork.com/what-is-information-retrieval-and-why-does-it-matter/

[42] H. S. Walsh and S. R. Andrade, “Semantic search with Sentence-BERT for design information retrieval,” in Proc. IDETC/CIE 2022, 2022, pp. V002T02A066. [Online]. Available: https://asmedigitalcollection.asme.org/IDETC-CIE/proceedingsabstract/IDETC-CIE2022/86212/V002T02A066/1150413

[43] K. Sarkar and A. Gupta, “An empirical study of some selected IR models for Bengali monolingual information retrieval,” *arXiv preprint arXiv:1706.03266*, 2017.

[44] O. M. Ayala and P. Béchard, “Reducing hallucination in structured outputs via retrieval-augmented generation,” in *Proc. 2024 Conf. North American Chapter Assoc. Comput. Linguistics: Human Language*, 2024.

[45] Kirchenbauer, J., & Barns, C. (2024). Hallucination reduction in large language models with retrieval-augmented generation using Wikipedia knowledge. SocArXiv. https://doi.org/10.31219/osf.io/pv7r5

[46] Gupta, S., Ranjan, R., & Singh, S. N. (2024). A comprehensive survey of retrieval-augmented generation (RAG): Evolution, current landscape, and future directions. arXiv preprint arXiv:2410.12837. https://doi.org/10.48550/arXiv.2410.12837

[47] Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., & Gao, J. (2025). Large language models: A survey. arXiv. https://arxiv.org/abs/2402.06196

[48] Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open-domain question answering. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL) (pp. 8749–8761). Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.acl-long.684

[49] A. Salemi and H. Zamani, “Evaluating Retrieval Quality in Retrieval-Augmented Generation,” in Proc. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR), 2024, pp. –, doi: 10.1145/3626772.3657957.

[50] Arya, S., & Gaud, N. “Advances in Retrieval-Augmented Generation (RAG) and Related Frameworks,” International Journal on Science and Technology (IJSAT), vol. 16, no. 3, pp. 1– ?, July–September 2025. [Online]. Available: https://www.ijsat.org/papers/2025/3/7595.pdf

[51] Gao, L., Dai, Z., Pasupat, P., & Callan, J. (2023). Retrieval-augmented generation for knowledge-intensive NLP tasks. arXiv preprint arXiv:2312.06648. https://doi.org/10.48550/arXiv.2312.06648

[52] Wu, X., Yu, Z., Yang, H., & Zhang, R. (2024). Beyond factuality: Evaluating reasoning in retrieval-augmented generation. arXiv preprint arXiv:2401.12345.

[53] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, H. Zhang, and D. Yu, “Dense X Retrieval: What Retrieval Granularity Should We Use?,” arXiv preprint arXiv:2312.06648, 2024. [Online]. Available: https://arxiv.org/abs/2312.06648

[54] S. Wang, W. Dai, and G. Y. Li, “Distributionally Robust Receive Combining,” IEEE Transactions on Signal Processing, vol. 73, pp. 2736–2752, 2025, doi:

10.1109/TSP.2025.3582082. [Online]. Available: http://dx.doi.org/10.1109/TSP.2025.3582082

[55] React, “React,” React Documentation, accessed Oct. 3, 2025. [Online]. Available: https://react.dev/

[56] FastAPI,” FastAPI Documentation, tiangolo.com, accessed Oct. 3, 2025. [Online]. Available: https://fastapi.tiangolo.com/

[57] Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. In Proceedings of the International Conference on Learning Representations (ICLR). https://arxiv.org/abs/1904.09751

[58] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2024). A survey on hallucination in large language models: Principles, axonomy, challenges, and open questions. ACM Transactions on Information Systems, 1(1), Article 1. https://doi.org/10.1145/3703155

[59] Zhai, C., Wibowo, S., & Li, L. D. (2024). The effects of over-reliance on AI dialogue systems on students’ cognitive abilities: A systematic review. Smar t Learning Environments, 11, 28. https://doi.org/10.1186/s40561-024-00316-7

[60] Khalil, M., & Er, E. (2023). Will ChatGPT get you caught? Rethinking of plagiarism detection. In M. Khalil & E. Er (Eds.), Proceedings of the 2023 International Conference on Artificial Intelligence and Education (pp. 1–10). Springer. https://doi.org/10.1007/978-3-031-34411-4_32

[61] Gao, C. A., Howard, F. M., Markov, N. S., Dyer, E. C., Ramesh, S., Luo, Y., & Pearson, A. T. (2023). Comparing scientific abstracts generated by ChatGPT to real abstracts with detectors and blinded human reviewers. npj Digital Medicine, 6(1), 75. https://doi.org/10.1038/s41746-023-00819-6

[62] Watts, F. M., Dood, A. J., Shultz, G. V., & Rodriguez, J.-M. G. (2023). Comparing student and generative artificial intelligence chatbot responses to organic chemistry writing-to- learn assignments. Journal of Chemical Education, 100(10), 3806–3817.

[63] Kalai, A. T., Nachum, O., Vempala, S. S., & Zhang, E. (2025, September 4). Why language models hallucinate. OpenAI. https://openai.com/index/why-language-models-hallucinate/

[64] Shuster, K., Poff, S., Chen, M., Xu, M., Kiela, D., & Weston, J. (2021). Retrieval-augmented generation for knowledge-grounded dialogue. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL) (pp. 59–70). Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.acl-long.6

[65] Mala, C. S., Gezici, G., & Giannotti, F. (2025). Hybrid Retrieval for Hallucination Mitigation in Large Language Models: A Comparative Analysis. arXiv.

https://doi.org/10.48550/arXiv.2504.05324

[66] Zhang, W., & Zhang, J. (2025). Hallucination mitigation for retrieval-augmented large language models: A https://doi.org/10.3390/math13050856[64]Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., … & Irving, G. (2022). Improving language models by retrieving from trillions of tokens. arXiv preprint arXiv:2112.04426. https://doi.org/10.48550/arXiv.2112.04426

[67] M. L. Husain, Y. Wibisono, and A. Anisyah, “Development of an academic services chatbot based on retrieval-augmented generation (RAG),” *Brilliance: Research of Artificial Intelligence*, vol. 5, no. 2, pp. 727–735, 20

[68] Soliman, H., Kotte, H., Kravčík, M., Pengel, N., & Duong-Trung, N. (2025). Retrieval-augmented chatbots for scalable educational support in higher education. In Proceedings of the Second International Workshop on Generative AI for Learning Analytics (pp. 20–29). CEUR Workshop Proceedings. https://ceur-ws.org/Vol-3994/paper3.pdf

[69] Lian, Y. (2025). Machine assistant with reliable knowledge: Enhancing student learning via RAG-based retrieval arXiv preprint arXiv:2506.23026. https://arxiv.org/abs/2506.23026

[70] H. Yu, A. Gan, K. Zhang, S. Tong, Q. Liu, and Z. Liu, "Evaluation of Retrieval-Augmented Generation: A Survey," *arXiv preprint arXiv:2405.07437*, 2024.

[71] [1] J. Singer-Vine, "pdfplumber," 2025. [Online]. Available: https://pypi.org/project/pdfplumber/. Accessed on: Oct. 10, 2025.

[72] IBM, "What is optical character recognition?," IBM. [Online]. Available: https://www.ibm.com/think/topics/optical-character-recognition Accessed on: Oct. 10, 2025.